

Simulation-Based Design of an Autonomous Multi-Robot System for Warehouse Kitting and Fulfillment

Group 7, Harshada Prasad Kulkarni, Aditya Abhang, Lawrence Farhat, Juan Ernesto Rodriguez, Marx I. Bertrand, Shlok Vijaykumar Ashtekar

1. Abstract

This project presents a design that is based on the simulation for an autonomous multi-robot warehouse kitting and fulfillment system. The intent is to demonstrate responsiveness of the production system under changing demand conditions. As such, this project addresses a common warehouse challenge where customer demand can shift quickly, yet kitting operations must still move parts reliably, maintain order accuracy, and avoid delays that disrupt downstream delivery preparation. To study this problem, the project models a coordinated robotic workflow in CoppeliaSim that begins with order entry and proceeds through part generation, part inspection and sorting, bin transport, and final placement into shipping trays. The simulated system includes a parts producer, a pick-and-place robot, two mobile robots, and a Cobot. Each system is assigned a defined role within the overall material-handling sequence. The producer generates parts according to demand, the pick-and-place robot transfers parts into the correct bins, the first mobile robot moves bins from the producer area to kitting, the second mobile robot supports downstream transport to delivery preparation, and the Cobot performs the final transfer of parts into shipping trays and empty bin removal. Signal-based coordination is used to synchronize handoffs between stations so that the system behaves as an integrated fulfillment process rather than a set of isolated robotic tasks.

The project also examines how simulation can be used to model motion, task logic, and to evaluate operational performance before physical deployment. System behavior is assessed using practical production measures such as throughput, cycle time, utilization, availability, and mean time to recovery. In addition, the simulation incorporates sensing, inspection logic, and selected fault scenarios to evaluate system resilience when normal operation is interrupted. These scenarios include pickup failures, material flow interruptions, and transfer delays that require safe recovery behavior and re-coordination among the robotic agents. This project provides a practical framework for studying autonomous warehouse kitting operations by combining robotic coordination, perception, material flow logic, and performance validation in a realistic simulation environment. The resulting concept demonstrates how a flexible multi-robot system can support more adaptive, reliable, and efficient fulfillment while offering a strong foundation for future refinement, testing, and possible physical implementation.

2. Motivation

Modern warehouse kitting and fulfillment systems are under increasing pressure to respond to changing demand, shorter delivery expectations, and stricter requirements for order accuracy. In these environments, the problem is not limited to moving parts from one station to another. The broader challenge is coordinating material flow in a way that remains efficient, flexible, and reliable even when order quantities, product mix, and operating conditions shift with little warning. Traditional kitting processes often rely on fixed workflows, manual handling, or isolated automation (Ho et al., 2025; Boysen & de Koster, 2025). Those approaches may work under stable

conditions, but they can become inefficient when demand changes suddenly or when disruptions occur. The result is often increased delay, idle time, bottlenecks, and reduced responsiveness across the fulfillment process (Park & Hong, 2025; Ho et al., 2025). This issue becomes especially important in just-in-time (JIT) environments, where interruptions in part movement can affect downstream operations and final delivery performance (Braunscheidel & Suresh, 2009).

This topic is important because responsiveness in warehouse operations has become a competitive requirement rather than a secondary improvement. Companies are expected to meet changing customer needs without sacrificing service quality, delivery speed, or operating efficiency (Dubey & Veeramani, 2024; Boysen & de Koster, 2025). Kitting systems are central to this challenge because they prepare and organize the materials required for specific orders before final delivery or assembly. If the kitting process cannot adapt to variation in demand, then the larger fulfillment system loses agility. Prior research has described this requirement in terms of supply chain agility, where firms must sense change early and respond quickly while maintaining a high level of service and competitive performance (Braunscheidel & Suresh, 2009). More recent studies have extended this idea to digitally connected and resilient supply networks, suggesting that responsiveness improves when strong information integration is combined with flexible processes and reconfigurable automation (Arman et al., 2025; Massari et al., 2025).

Several efforts have already been made to address this problem. At the research level, prior work has emphasized agile supply chains, digital integration, resilience, and the use of automation to improve responsiveness (Leung, 2025; Dubey & Veeramani, 2024). In operational settings, warehouse systems have incorporated conveyors, pick-and-place robots, mobile robots, Cobots, and sensor-based inspection systems to improve local efficiency and to reduce manual effort. However, these technologies are often implemented as separate subsystems rather than as a coordinated end-to-end fulfillment process (Leung, 2025; Ho et al., 2025). Although there have been some advances in this space, a clear gap remains. Much of the existing discussion around warehouse agility and automation stays at a high level. The literature explains why flexibility, digitalization, and robotic assistance matter, but it does not always show how multiple robotic agents should be coordinated across the full kitting and fulfillment process under changing demand conditions (Zhen et al., 2023; Keung et al., 2023). In many cases, automation improves one part of the process without ensuring reliable performance across the entire system. There is still a need for a validated system-level framework that links upstream part generation, intermediate pick-and-place actions, bin transport, final tray loading, and fault recovery within one integrated operating environment. That gap is especially important because effective fulfillment depends not only on individual robot performance, but also on reliable handoffs, synchronized task execution, and the ability to recover safely from disruptions.

This project proposes to address that gap through the simulation-based design of an autonomous multi-robot warehouse kitting and fulfillment system using CoppeliaSim. The system models the full workflow from order entry to delivery preparation through the coordinated operation of a parts producer, a pick-and-place robot, two mobile robots, and a Cobot. Instead of evaluating each component in isolation, the project studies how these elements interact as part of one integrated process. The simulation environment supports the testing of robot motion, signal-based coordination, pickup and release logic, bin transport, inspection behavior, and task sequencing in a controlled but realistic environment. It also allows the team to introduce faults such as missed pickups, dropped bins, navigation issues, and conveyor interruptions to study fault detection and safe recovery behavior. By evaluating throughput, cycle time, utilization,

availability, and mean time to recovery, the project aims to demonstrate how a coordinated multi-robot architecture can improve responsiveness, reduce fulfillment delays, and support more agile warehouse operations under variable demand conditions.

3. Problem & Background

Warehouse kitting and fulfillment systems are expected to move materials quickly, accurately, and reliably even when customer demand changes with little warning. In practice, this means that orders must be received, parts must be generated or retrieved, items must be inspected and sorted correctly, and materials must reach downstream delivery preparation without delay. The problem is not only one of transportation. It is a coordination problem across multiple stages of material handling. When that coordination breaks down, warehouses experience idle time, bottlenecks, delayed order completion, and reduced responsiveness. These issues become more significant in JIT operating environments, where interruptions in part flow can affect service performance and delivery readiness (Braunscheidel & Suresh, 2009).

This topic is important because fulfillment agility has become a basic operational requirement. Warehouses are no longer judged only by storage capacity or labor efficiency. They are also judged by how quickly they can respond to demand variability while maintaining order accuracy and continuity of flow. Prior research describes this broader requirement as supply chain agility, meaning that firms must sense changes and respond quickly without sacrificing performance (Braunscheidel & Suresh, 2009). More recent studies extend that view by showing that digital connectivity, flexible automation, and reconfigurable logistics systems are important enablers of responsiveness and resilience in modern supply networks (Arman et al., 2025; Massari et al., 2025).

A substantial body of work has already addressed warehouse automation and robotic fulfillment. Reviews of automated and robotized warehouse systems show that robotic handling technologies are attractive because they use space efficiently, offer flexibility under varying demand, and support continuous operation. These research endeavors also identify robotic mobile fulfillment systems as an important direction in warehouse design. More focused work on robotic mobile fulfillment systems shows that warehouse performance depends not only on the presence of robots, but also on how orders, pods, workstations, and robots are assigned and coordinated across the system. In other words, deployment and scheduling decisions can strongly affect order fulfillment time and overall system efficiency. Studies of robotic mobile fulfillment through simulation further show that cyber-physical approaches can support JIT material transfer and improve evaluation of coordinated robotic operations before implementation.

The attached project materials align with this direction, but they focus on a specific warehouse kitting workflow rather than a generic automated warehouse. The project defines a coordinated system that begins with order entry and continues through part generation, part inspection and sorting, bin transport, and final placement into shipping trays. The simulated environment includes a parts producer, a pick-and-place robot, two mobile robots, and a cobot, each assigned a distinct role within the material-handling sequence. The producer continuously generates parts, the pick-and-place robot sorts parts into the proper bins, Mobile Robot 1 moves bins to kitting, Mobile Robot 2 supports downstream transfer, and the cobot performs final tray loading and empty bin removal. Signal-based coordination is used so that these agents operate as one fulfillment process rather than as isolated robotic tasks.

Even with these advances, an important gap remains. Much of the literature explains the

value of warehouse automation, robotic fulfillment, or digital integration at a broad level, but less attention is given to how multiple robotic agents should be coordinated across the full kitting workflow under changing demand conditions and possible disruptions. Existing studies often emphasize order picking, robot scheduling, or local efficiency gains, while system-level questions about synchronized handoffs, inspection-driven routing, bin movement, recovery from transfer failures, and downstream preparation are less fully integrated into one framework. Simulation studies of robotic mobile fulfillment systems have shown the importance of task allocation and operational coordination, especially in constrained warehouse settings. However, there is still a need for a validated environment that connects upstream part production, sorting, mobile transfer, final placement, and resilience logic in one continuous process. That gap is central to this project because real warehouse performance depends not only on robot capability, but also on dependable interactions between robots, sensors, buffers, and downstream tasks.

This project addresses that gap through the simulation-based design of an autonomous multi-robot warehouse kitting and fulfillment system in CoppeliaSim. Rather than evaluating each subsystem independently, the project studies the complete workflow from order entry through delivery preparation. The simulation is used to model robotic motion, part inspection, bin handling, signal-based coordination, and task sequencing in a realistic environment. It is also used to evaluate operational performance before physical deployment using measures such as throughput, cycle time, utilization, availability, and mean time to recovery. In addition, the project incorporates sensing, inspection logic, and selected fault scenarios such as pickup failures, material-flow interruptions, and transfer delays. This is consistent with recent research showing that simulation-based cyber-physical fulfillment environments can support more rigorous testing of coordinated robot actions and JIT transfer logic before deployment. The project provides a practical framework for examining how a flexible multi-robot architecture can improve warehouse responsiveness, reduce fulfillment delays, and strengthen operational resilience under variable demand conditions by combining robotic coordination, material-flow logic, and performance validation.

4. Method

This study used a simulation-based method to design and evaluate an autonomous multi-robot warehouse kitting and fulfillment system in CoppeliaSim. The method is intended to represent the full order-fulfillment flow from order entry through final placement of parts into shipping trays. The modeled system included a parts producer, a pick-and-place robot, two mobile robots, and a cobot operating within a shared control architecture. The simulation is used not only to model robotic motion and task execution, but also to evaluate system coordination, material flow, and operational performance before physical implementation. This approach is consistent with the objective of studying how a flexible multi-robot architecture can improve responsiveness, reduce fulfillment delays, and support more resilient warehouse operations under changing demand conditions.

4.1 Simulation Environment and Modeling Scope

The system is implemented in CoppeliaSim as a coordinated warehouse kitting model with separate simulated work areas for the producer, kitting, delivery preparation, and bin return stages. The modeled workflow begins when the user enters the requested quantities of green, blue, yellow, and red parts through an ordering interface. From that point, the simulation executes a sequence in which parts are produced, inspected and sorted into bins, transferred from the producer area to

kitting, moved from kitting to delivery preparation, and finally unloaded into shipping trays by the Cobot.

The simulation emphasizes coordination logic and task sequencing rather than high-fidelity dynamic analysis. Each robot is assigned a defined functional role within the material-handling process. The producer continuously generates parts; The pick-and-place robot transfers parts into the correct color bin or scrap bin based on inspection logic; Mobile Robot 1 moves ready bins from the producer area to kitting; Mobile Robot 2 transports bins from kitting to delivery preparation. The Cobot unloads parts from the delivery bins into the appropriate shipping trays and removes empty bins as required. This scope allows the study to focus on how robot interactions and handoffs affected system-level performance.

4.2 Control Architecture

The control structure is implemented as a coordinator-based architecture in which a single central controller manages the shared system state while each robot executes a separate local state machine. This structure was selected to more effectively support the project sequence in a controlled and traceable manner. Under this design, order entry, part generation, producer bin filling, transfer to kitting, transfer to delivery preparation, and Cobot unloading are all governed through one supervisory layer. An advantage of this approach is the architecture reduces integration ambiguity because each robot receives a specific work request, executes its assigned task, and returns a completion acknowledgement before the coordinator advances the system state.

The system architecture, shown in Figure 1, separates system-level decision making from robot-level execution. The Order Console collects operator input and publishes the requested quantities. The coordinator maintains the active order, tracks bin status, assigns work, manages inter-robot handshakes, and determines when the order is complete. The pick-and-place robot, Mobile Robot 1, Mobile Robot 2, and the Cobot each operated as local state machines that responded only to the assigned request signals. This separation improves robustness and simplifies debugging because the robots do not need to infer global system status from multiple sources. Instead, the coordinator owns the shared system truth, while each robot manages only its own execution logic.

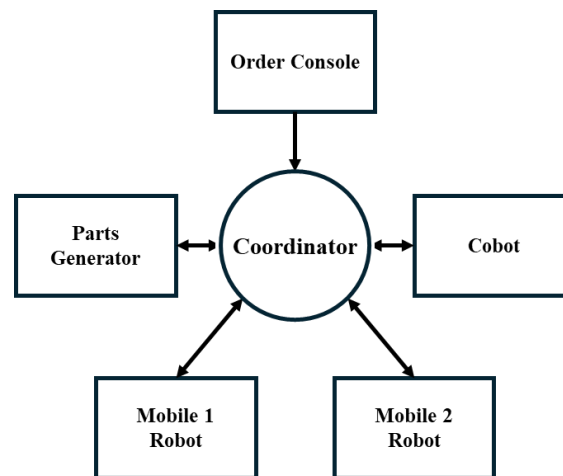


Figure 1. Coordinator-based Architecture

4.3 Shared Signal Protocol

System integration relies on a shared-signal communication protocol. The Order Console publishes the requested quantities through the following signals: `order_green`, `order_blue`, `order_yellow`, and `order_red`. Once an order is submitted, the signals `order_ready` and `system_enabled` are enabled. The coordinator then copies the submitted values into `remain_green`, `remain_blue`, `remain_yellow`, and `remain_red` so that the original order quantities remain unchanged while the active remaining quantities are updated during fulfillment. This distinction allows the system to preserve the original request while separately managing order progress.

Each robot follows a *request-busy-done handshake* convention. A *request-busy-done handshake convention* is a control protocol that is used to coordinate an initiator and a responder when one side requests an operation and the other side performs it asynchronously. For example, the pick-and-place robot responds to `req_pick_place`, enables `pick_place_busy` while the transfer is in progress, and then enables `pick_place_done` after completion. The same pattern is used for Mobile Robot 1, Mobile Robot 2, and the Cobot. This handshake protocol prevents overlapping claims on the same work item and it gives the coordinator a reliable mechanism for sequencing operations and acknowledging task completion. In effect, the signal protocol served as the control interface that links the independent state machines into one integrated fulfillment process.

4.4 Coordinator Logic

The coordinator serves as the supervisory layer for the full warehouse flow. At the beginning of each new order, it initializes the remaining required quantities and resets the active workflow signals. During simulation, it continuously evaluates the current system state and dispatched work according to the next required operation. The coordinator logic is organized around the availability of needed parts, the readiness of bins, the location of bins in the workflow, and the status of each robot.

First, the coordinator determines whether additional parts of each color are required. If a color is still needed and a producer bin of that color is available, the coordinator issues a pick-and-place request and identifies the destination producer bin. Producer bins are considered ready for transfer when they reach the maximum capacity of one part. Second, when a producer bin becomes ready, the coordinator requests Mobile Robot 1 to transfer that bin from the producer area to the kitting area. After the transfer is acknowledged, the coordinator updates the bin record so that the bin is no longer considered available at the producer and is instead available in kitting. Third, when a bin in kitting is needed to satisfy the remaining order, the coordinator reserves that bin and requests Mobile Robot 2 to move it to the delivery preparation area. This reservation logic prevents conflicting requests and supports fulfillment across multiple bins if the order requires it. Fourth, when a delivery bin is present, the coordinator requests the Cobot to unload the parts to the shipping tray. The remaining order quantity is decremented only after the cobot completes unloading because unloading to the tray represents the true fulfillment point. This prevents false completion states in the cases where parts have been produced or transported but have not yet reached the final tray. Finally, the coordinator sets `order_complete` only when all remaining quantities are zero and no robot is still executing or awaiting acknowledgement. This definition ensures that the system declares completion only after the entire workflow has finished and all of the pending actions have been resolved. This flow is depicted in Figure 2 below.

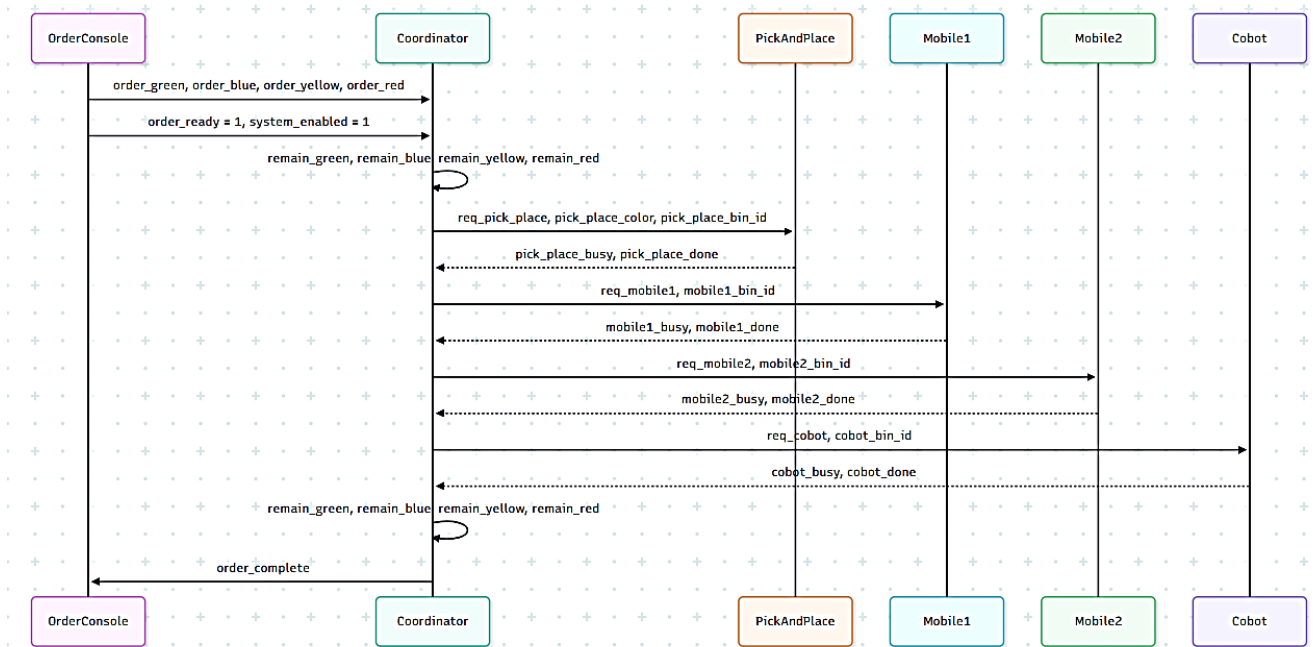


Figure 2. End-to-End Bin Fulfillment Flow (Coordinator Controlled)

Main flow (high-level)

The following sequence summarizes the simplified coordinator-driven workflow used in the model. It shows how order signals are issued, how work is dispatched to each robot, and how remaining quantities are updated until the order is complete.

1. The Order Console sends `order_green`, `order_blue`, `order_yellow`, and `order_red` to the Coordinator, then sets `order_ready = 1` and `system_enabled = 1`.
2. The Coordinator initializes `remain_green`, `remain_blue`, `remain_yellow`, and `remain_red` from the submitted order values.
3. If parts are still needed, the Coordinator sends `req_pick_place` with `pick_place_color` and `pick_place_bin_id` to the Pick-and-Place robot. The Pick-and-Place robot responds through `pick_place_busy` and `pick_place_done`.
4. When a producer bin is ready for movement, the Coordinator sends `req_mobile1` with `mobile1_bin_id` to Mobile Robot 1. Mobile Robot 1 responds through `mobile1_busy` and `mobile1_done`.
5. When a kitting bin is needed for the next stage, the Coordinator sends `req_mobile2` with `mobile2_bin_id` to Mobile Robot 2. Mobile Robot 2 responds through `mobile2_busy` and `mobile2_done`.
6. When a delivery bin is ready to be unloaded, the Coordinator sends `req_cobot` with `cobot_bin_id` to the Cobot. The Cobot responds through `cobot_busy` and `cobot_done`.
7. After Cobot completion, the Coordinator updates `remain_green`, `remain_blue`, `remain_yellow`, and `remain_red` based on the fulfilled parts.
8. The Coordinator sets `order_complete` only after all remaining quantities are zero.

4.5 Robot State-Machine Logic

Each robot is implemented as an independent local state machine that waits for a coordinator request, executes the assigned operation, and then returns a completion acknowledgement. The simulated hardware includes a producer object, a UR10 pick-and-place arm, a Pioneer3DX mobile base paired with a UR5 arm for Mobile 1, a KUKA youBot mobile manipulator for Mobile 2, and a UR10 arm for the Cobot. Robots do not make global task-allocation decisions. That responsibility remains with the coordinator so that robot logic can remain simple, localized, and easier to validate. The pick-and-place robot waits for a request from the coordinator, reads the target color and destination producer bin, performs the part transfer, and then signals completion. It does not decide which color to pick or which producer bin to fill. Mobile Robot 1 waits for a request to move a ready producer bin, reads the bin identifier, executes the producer-to-kitting transfer, and then returns a done signal. Mobile Robot 2 waits for a request to move a reserved bin from kitting to delivery preparation, reads the requested bin identifier, executes the transfer, and acknowledges completion. The Cobot waits for a request to unload a delivery bin, reads the bin identifier, unloads the parts to the shipping tray, and returns a done signal so that the coordinator can update the remaining quantities in one location. This arrangement maintains consistency between local execution and global order tracking.

4.5.1 Part Producer

The Part Producer serves as the upstream source of parts and is responsible for generating items based on the active order. In the simulation, the producer reads the order signals (`order_green`, `order_blue`, `order_yellow`, and `order_red`) and converts them into internal pending counts. Once the system is enabled and an order is released, the producer transitions from an idle state to an active, demand-driven mode. This allows part generation to reflect actual order requirements rather than continuous random spawning, which improves the realism of the overall workflow.

The producer logic maintains a local pending table that tracks how many parts of each color still need to be generated. At each spawn interval, the script selects one of the remaining required part types and creates a new instance by copying a template object. The spawned part is positioned at a defined spawn point, assigned a unique identifier, labeled with its part type, and converted into a dynamic object so it can interact with the conveyor and downstream systems. After each successful spawn, the corresponding pending count is reduced. This continues until all required parts have been generated, at which point the producer signals order completion. A key feature of the producer method is the use of controlled spawn timing and area validation. The script introduces a randomized interval between spawns to avoid unrealistic bursts of part generation. Before creating a new part, the producer checks whether the spawn area is clear using a defined radius. If another object is detected within that space, spawning is delayed until the area becomes free. This prevents overlapping objects, reduces physics instability, and ensures smoother interaction with the conveyor and vision system. This behavior reflects a practical constraint found in real production systems, where upstream release is often gated by downstream availability. The producer is tightly integrated with the conveyor, camera, and pick-and-place robot. Once a part is spawned, it becomes part of the material flow and is transported, identified, and sorted by downstream subsystems. The producer itself does not control these stages, but its timing and output directly affect their performance. This highlights an important system-level characteristic: successful operation depends on coordinated interaction across multiple subsystems rather than independent execution. In a real implementation, this coordination would be managed through a

supervisory PLC using permissives, signals, and sequence logic to regulate part release and downstream readiness. The script also includes state management logic to handle order loading and completion. When a new order is received, the producer initializes the pending counts and sets an active state. When all required parts have been generated, it signals completion and resets the order-ready condition. This ensures that the producer does not continue generating parts beyond what is required and maintains alignment with the overall fulfillment process. Overall, the Part Producer establishes the starting point for the entire workflow and demonstrates how upstream production can be coordinated with downstream handling through signal-based control. It shows that even a simple generation process must incorporate demand tracking, spatial checks, and timing control to support stable and realistic system behavior

4.5.2 Pick-and-Place Robot

The Pick-and-Place robot performs part inspection and sorting between the producer and the bin staging area. In this model, the Pick-and-Place robot is represented by a UR10 robotic arm, which provides six-axis manipulation for moving parts from the pickup location to the appropriate color bin or scrap location. In the simulation, the robot operates as a local state machine that waits for a part to arrive at the pickup location, moves to an inspection pose, and triggers a vision-based classification process. The script uses a camera signal interface in which the robot requests an image (`takeImage`), waits for a classification response (`colorReady`), and reads the detected part color (`detectedColor`). Once the part is classified, the robot activates a vacuum gripper, lifts the part, and moves it to the corresponding drop location based on the detected color. Each color is mapped to a predefined drop pose, allowing consistent placement into the correct producer bin.

The robot motion is implemented using inverse kinematics with defined pick, intermediate, and drop poses to ensure smooth and collision-aware movement. Intermediate poses are used to transition safely between pickup and placement locations, which improves repeatability and reduces the risk of interference with surrounding objects. After placing the part, the robot releases the vacuum, returns to the pickup position, and signals the conveyor to resume operation. This sequence ensures continuous material flow while maintaining synchronization with upstream and downstream processes. Overall, the Pick-and-Place robot demonstrates how inspection, classification, and controlled manipulation can be integrated into a signal-driven workflow to support reliable part sorting within the coordinated system.

4.5.3 Mobile 1 Robot

Mobile 1 performs the upstream bin-transfer task between the producer area and the kitting area using a Pioneer3DX mobile base for transport and a UR5 robotic arm for local bin pickup and drop-off. In the simulation, the robot is modeled as an autonomous mobile robot that moves between predefined park positions for pickup and drop-off. The script defines a pick parking location and a drop parking location, then organizes the mission into two phases: a pickup phase and a drop-off phase. During execution, the robot waits briefly at startup, drives toward the current target position, and stops when it reaches the defined position tolerance. This approach provides a simple and repeatable method for moving the robot between the producer-side pickup area and the kitting-side drop area.

The Mobile 1 script uses differential-drive control by commanding the left and right wheel motors based on the robot's current position, target position, and heading error. The robot continuously calculates the distance to the target and adjusts its forward and turning velocities to steer toward the next waypoint. Once it reaches the parking position, it stops and sends a signal to the robot arm to begin the local pickup or drop-off action. For example, the script sends a pickup signal at the producer-side park position and a drop-off signal at the kitting-side park position. The

mobile base then waits for an arm completion signal before advancing to the next phase. This structure separates mobile navigation from arm handling while still allowing the two subsystems to operate as one coordinated task.

The Mobile 1 logic also includes practical debugging and sequencing features. The script prints periodic status messages showing the active phase, robot position, target location, and remaining distance to the waypoint. It also holds the robot stopped while waiting for the arm to complete its task, which prevents motion from continuing before the bin-handling operation is finished. Overall, Mobile 1 serves as the transport link between the producer area and the kitting area. Its logic demonstrates how mobile navigation, parking-position control, and handoff signaling can be combined to support reliable upstream bin movement within the larger coordinator-based fulfillment workflow.

4.5.4 Mobile 2 Robot

Mobile Robot 2 performs the downstream transfer task after a bin has been moved from the producer area to the kitting area. Mobile Robot 2 is represented by a KUKA youBot mobile manipulator, which combines a mobile base with a five-degree-of-freedom robotic arm and gripper for bin pickup, transport, and placement. In the simulation, Mobile Robot 2 waits for a request from the supervisory coordinator, reads the assigned `mobile2_bin_id`, looks up the corresponding bin color and location information, and then moves the bin from the kitting pickup point to the appropriate delivery drop-off point. This supports the larger fulfillment sequence by connecting the kitting stage to the final delivery preparation area.

The Mobile 2 script uses a local lookup table to map each assigned bin ID to its corresponding color, kitting pickup dummy, and delivery drop dummy. This design keeps the robot-side logic simple because the robot does not decide which bin is needed or where the order stands. Those decisions remain with the supervisory coordinator. Mobile Robot 2 only accepts the assigned task, validates the bin ID, finds the required scene objects, executes the local motion sequence, and reports completion through the `mobile2_busy` and `mobile2_done` signals. This follows the same PLC-style command-and-status structure used throughout the system. The script combines mobile base motion with arm and gripper control. The mobile base uses heading targets and drive primitives to travel to the kitting pickup row, face the bin shelf, approach the assigned bin, back out, travel to the delivery row, and approach the drop-off point. The arm sequence then positions the gripper to pick up the bin, carry it safely, and place it at the delivery location. The gripper uses separate open and close targets for the left and right gripper joints, and a grip-lock function holds the bin during transport. These details were important because the bin transfer task required both navigation accuracy and reliable object handling. A key part of the Mobile 2 method was tuning the pickup sequence around the physical geometry of the staging area and bin wall. The script uses several defined arm poses, including a tucked pose, carry pose, pre-lip pose, reach-above pose, reach-high pose, lift-start pose, and placement poses. These poses guide the robot through a controlled pickup and placement motion. In particular, the reach-above and reach-high poses were tuned so the gripper could descend onto the bin wall with minimal lateral drift. This helped improve repeatability and reduced the risk of the bin slipping or being pulled off-axis during pickup. The Mobile 2 logic also includes practical safeguards for motion and debugging. The robot uses timeout limits during rotation and travel, a table-avoidance check near the drop-off area, periodic diagnostic messages, and error handling when assigned bin IDs or required dummy objects cannot be found. These features made the script easier to test and helped identify integration problems during simulation. Overall, Mobile Robot 2 serves as the downstream

transport link in the system and demonstrated how local navigation, arm manipulation, gripper control, and supervisory signaling must work together for reliable bin delivery.

4.5.5 Cobot

The Cobot performs the final unloading operation after Mobile Robot 2 delivers a bin to the delivery preparation area. In the simulation, the Cobot is represented by a UR10 robotic arm that waits until a delivery bin is detected, then follows a defined pick-and-place sequence to transfer the part from the bin to the correct shipping tray. Following each unloading operation, the cobot picks up and discards the empty to clear out the delivery area for the next part order. This task is controlled locally by the Cobot script, while the overall timing of the operation remains part of the larger coordinator-based workflow. This is consistent with the project architecture, where the supervisory controller manages sequence timing, and each robot executes its assigned local motion task.

The Cobot script uses inverse kinematics to control the robot arm motion. The program defines a home pose, safe transit pose, pick approach poses, pick poses, drop approach poses, and tray drop poses. A similar architecture is used to discard the empty bin after each loading task, using empty bin approach and pick poses as well as bin discard approach and drop poses. This structure allows the Cobot to move through safe intermediate positions before approaching a bin or tray. The use of approach poses is important because it reduces the chance of collision and provides a more repeatable motion path during pickup and placement. The script also uses separate fast and slow-motion profiles. Faster motion is used for open travel between safe positions, while slower motion is used near the pickup and drop-off points where precision is more important. Object detection is handled through proximity sensors located at the delivery stations. When a sensor detects a new bin, the script latches that sensor input and assigns the corresponding station index to the active task. This prevents the Cobot from repeatedly responding to the same detection event while the bin remains in place. After a valid sensor trigger is received, the Cobot moves from home to the safe transit pose, approaches the detected bin, lowers to the pickup pose, activates the vacuum gripper, lifts the part, moves to the matching tray location, releases the vacuum, and then returns to home. The vacuum gripper is controlled through a signal that turns suction on during pickup and off during release. This allowed the simulation to represent the final part-transfer step without requiring direct manual intervention. The script also prints progress messages during execution, which supports debugging and helps confirm the current step of the Cobot sequence during simulation testing. Overall, the Cobot logic completes the last stage of the fulfillment process by converting a delivered bin into a completed tray-loading action. This makes the Cobot the final confirmation point for order fulfillment in the modeled workflow.

4.6 Order Console and Operator Input

The Order Console is implemented as a CoppeliaSim dialog that allows the operator to enter the requested quantities of green, blue, yellow, and red parts. When the user selects Submit Order, the console publishes the requested quantities to the order signals and enables `order_ready` and `system_enabled` signals. The console also provides a reset function that clears the order values and resets the handshake signals so that stale requests will not propagate into the next simulation run. This arrangement establishes a clear boundary between operator input and execution control. The console captures user intent, but all fulfillment decisions are performed by the coordinator after the order is released.

4.7 Experimental Procedure

Experimental trials are conducted by initializing the system to a cleared state, entering the requested part quantities through the Order Console, and allowing the simulated workflow to proceed until either all requested quantities have been unloaded into the shipping trays or a defined interruption prevented a normal completion. Each run follows the same general sequence: order submission, part generation, sorting into producer bins, transfer to kitting, transfer to delivery preparation, and final tray loading. Between trials, the system signals and order values are reset to ensure that each run begins from the same initial state. This procedure allows the coordination logic and handoff behavior to be observed repeatedly under the same starting conditions.

The study design supports evaluation under both normal and interrupted operating conditions. In normal runs, the system is allowed to execute the full order sequence without injected faults. In selected disrupted runs, faults such as pickup failures, material-flow interruptions, and transfer delays are introduced to examine whether the signal protocol and task coordination logic could support safe recovery and continued order fulfillment. These disrupted runs are included because the project explicitly aimed to study not only efficiency but also resilience and re-coordination among robotic agents when normal operation is interrupted.

4.8 Performance Measures

System behavior is evaluated using practical production measures identified in the project objectives. These included throughput, cycle time, utilization, availability, and mean time to recovery. Throughput is defined as the number of fulfilled parts or completed kits processed during a simulation run. Cycle time is defined as the elapsed simulated time from order submission to final unloading of the last required part into the shipping tray. Utilization refers to the proportion of time that each robot spends actively executing assigned tasks relative to the total simulation time. Availability refers to the proportion of time that the system remains able to execute the required tasks without being blocked by fault conditions or incomplete handshakes. Mean time to recovery is defined for disrupted runs as the average simulated time required for the system to restore the workflow after a fault condition occurred. These measures were selected because they captured both production efficiency and operational resilience.

4.9 Modeling Assumptions and Scope Limits

Several simplifying assumptions were used in the model to keep the study focused on system coordination. Producer bins were treated as having a maximum capacity of one part. The coordinator-based architecture assumed a single active system-level controller rather than a distributed negotiation scheme among robots. Robot scripts were designed to execute only coordinator issued requests, which reduced local autonomy but improved traceability and consistency of task sequencing. The model also assumed that order completion should be defined at the point of final tray unloading rather than at earlier stages such as part production or bin movement. In the simplified implementation, empty bins are logically reset to the producer location after Cobot unloading rather than being physically returned by a separate transport routine. These assumptions were appropriate for a study centered on coordination, material flow, and fulfillment validation, but they also define the limits of generalization for future work. This method provides a structured means of studying how centralized coordination, local robot state machines, and signal-based handshakes could be used to manage a multi-stage robotic warehouse process. By combining system architecture, task sequencing, operator input, fault scenarios, and production-oriented performance measures within one simulation environment, the study

established a defensible basis for evaluating autonomous warehouse kitting and fulfillment before physical deployment.

5. Main Findings

The main finding of this project is that a multi-robot warehouse kitting and fulfillment process can be modeled effectively through a centralized supervisory control structure that resembles a real programmable logic controller (PLC) based architecture. In the simulation, the coordinator served as the supervisory layer for the overall system. In a real implementation, this role would align with a supervisory PLC or higher-level cell controller that monitors system state, issues commands to individual devices, receives status feedback, and determines when each step in the process may proceed. Under this structure, the robots do not exchange signals directly with one another. Instead, each asset interacts with the supervisory layer through defined signals, while the supervisory layer manages the full sequence of operation. This is an important result because it places the project within a realistic industrial controls framework rather than a purely abstract simulation scheme. A major finding was that this supervisory structure provided a workable method for coordinating the full order-fulfillment process. In the simulation, the workflow begins with order entry and continues through part generation, sorting, producer bin filling, transfer to kitting, transfer to delivery preparation, and final unloading into shipping trays. The coordinator linked these stages together and maintained the shared order state throughout the process. In practical terms, this is similar to how a supervisory PLC would coordinate a multi-station automated cell. The study showed that a clear system-level controller improves traceability, simplifies sequencing, and keeps the material flow organized from the initial order request through final fulfillment.

One important finding concerned the part producer. The simulation demonstrated a useful shift from random part generation to demand-driven production. Rather than allowing parts to appear continuously without regard to order need, the producer could be tied to order signals so that part creation reflected actual demand. This made the upstream process more realistic and better aligned with the rest of the fulfillment sequence. It also showed that production logic should be coordinated with downstream readiness rather than treated as an isolated function. In a real-world automated system, that kind of behavior would normally be governed by a supervisory controller using permissives and release conditions. The simulation captured that idea in a simple but meaningful way. The project also showed that the part producer depended heavily on its integration with the conveyor, camera, and pick-and-place robot. Proper operation required the producer to generate a part at the correct time, the conveyor to present the part correctly, the camera to identify the relevant part condition or color, and the pick-and-place robot to sort the part into the proper bin. This demonstrated that system success depended not only on individual robot motion, but also on coordinated timing and signal discipline across several interacting subsystems. That is a significant result because it reflects how real automation systems operate. The value of the architecture came from coordinated interaction across stations, not from isolated task execution alone.

Another strong finding was the usefulness of the request-busy-done signal structure. In the model, the coordinator issued a request to a robot; the robot set a busy signal while the task was active and then returned a done signal on completion. This is similar to the handshake pattern commonly used between a PLC and a robot or other controlled device. The study showed that this logic was sufficient to coordinate the pick-and-place robot, Mobile Robot 1, Mobile Robot 2, and the Cobot in an orderly sequence. That result is important because it demonstrates that the

simulated control concept is consistent with a familiar industrial control pattern and can support reliable inter-station handoffs without direct robot-to-robot communication. The Mobile Robot 2 work contributed another important finding. Downstream transport was not merely a navigation task. The simulation showed that Mobile Robot 2 can move from a home position to the kitting area, pick up an assigned bin with its gripper after receiving a coordinator request, transport the bin, and place it at the downstream delivery location so the next robot can complete the kit-loading process. Reliable operation depended on precise interaction between the mobile base, arm, gripper, bin, staging area, table, and surrounding path geometry. The most difficult part of the task was grasping the narrow front ledge of the bin. Bin wall thickness, part weight, gripper opening range, gripper strength, and arm torque limits all affected whether the pickup could be repeated reliably. Slower arm motion, a final approach point directly above the front bin edge, and a precise stopping distance from the staging rack were important for consistent pickup. This finding reinforced a broader lesson from the project: the supervisory architecture can organize the process, but repeatability still depends on disciplined local operating conditions and well-defined physical interfaces at each station. The simulation also highlighted practical constraints related to spawning and physical stability. Spawn timing and collision avoidance required checks to confirm that the spawn area was clear before a new part was generated. Without that logic, the upstream process could become unstable or interfere with downstream flow. The study also showed that complex polygonal geometries caused unstable physics behavior, which led to the use of simpler part shapes such as cuboids and cylinders. Although this reduced geometric realism, it improved simulation stability and made the model more dependable for studying coordination, sequencing, and handoff logic. This was a useful engineering tradeoff because the goal of the project was to validate the system architecture rather than to optimize detailed contact physics.

Another finding was that order progress and completion were more reliable when managed by the supervisory layer rather than by individual devices. In the model, the coordinator maintained the remaining required quantities for each color and updated those values only after final unloading into the shipping tray. This kept fulfillment status aligned with the true completion point of the process. It also avoided the false completion of states that could arise if upstream production or transport were treated as fulfillment on their own. This is an important result because it shows that centralized order tracking improved consistency and preserved the integrity of the system state across the full workflow.

Taken together, these findings show that the project succeeded in demonstrating a realistic supervisory control concept for warehouse kitting and fulfillment. The strongest contribution is not simply that several robots and subsystems were placed in one simulation. It is that the workflow was organized in a way that resembles how an actual automated cell or warehouse subsystem would be coordinated in practice. The coordinator can be understood as a supervisory PLC layer, while the robots, conveyor, camera, and producer behave as locally controlled assets that respond to supervisory commands and return status signals. Under that interpretation, the project provides a credible early-stage model of how centralized industrial control can coordinate autonomous material handling across multiple stations.

6. Evaluation

The project should be evaluated primarily as a simulation-based proof of concept for coordinated warehouse fulfillment under a supervisory PLC-style control architecture. By that standard, it is a strong design. One of its main strengths is the clarity of the control structure. The coordinator acts as the supervisory layer for the overall system, much like a real PLC or higher-

level industrial controller would in practice. Robots and other subsystems do not need to communicate directly with one another. Instead, they respond to supervisory commands and return defined status signals. This makes the architecture realistic from an industrial controls perspective and gives the system a clear chain of coordination.

A second strength is the disciplined use of signal-based coordination. The signal-driven approach worked effectively for order-based production and staged handoff, but it also revealed the need for strict consistency in signal naming, timing, and interpretation across all subsystems. In a system that includes a producer, conveyor, camera, robotic arm, mobile robots, and a cobot, even a small mismatch in signal usage can create integration problems or incorrect sequence behavior. This is a meaningful result because it reflects the reality of industrial control integration. The simulation did not merely show that signaling can work. It also showed that successful coordination depends on well-defined interfaces and tight synchronization.

The part producer portion of the project evaluated well because it supported order-based production rather than uncontrolled random spawning. The system successfully produced and processed parts in a way that reflected released demand. At the same time, it exposed the challenge of integrating the producer with the conveyor, camera, and pick-and-place robot. As more subsystems had to act in sequence, debugging and coordination became more demanding. That is not a weakness in the study. It is one of its useful outcomes. The simulation revealed where integration complexity naturally appears in a real automated process and allowed those issues to be studied in a controlled environment before physical implementation.

The Mobile Robot 2 work added a similar evaluation point at the downstream end of the system. The robot successfully combined mobility and manipulation, but only after multiple programming iterations and fine tuning. The task required careful adjustment of stopping distance, arm approach, claw timing, grasp range, staging height, table height, bin properties, carry angle, and path geometry. This showed that downstream transfer and placement are interface-heavy tasks rather than simple motion commands. The evaluation also identified practical failure modes. If the gripper jaws were not aligned correctly, the jaw could strike the part in the bin and knock it out during pickup. The robot arm could also contact the staging post, which suggests that a lower post height or revised staging design would reduce interference risk. During placement, the carried bin had to remain high enough to avoid clipping nearby bins, but not so high that the part became unstable and fell out. These findings show that the most difficult implementation effort may not be in the high-level sequence itself, but in the detailed matching of robot motion, object handling, and station geometry.

The project also evaluates positively in terms of methodological fit. The report set out to study responsiveness, coordination, and resilience in a warehouse kitting process under changing demand conditions. The experimental structure matched that purpose. Normal runs allowed the team to observe whether the order could be completed from start to finish, while disrupted runs allowed the team to examine system behavior when expected flow was interrupted. The selected performance measures were also appropriate. Throughput and cycle time addressed production efficiency, utilization and availability addressed operational behavior, and mean time to recovery addressed resilience. This made the study method well aligned with the research objective. At the same time, the limits of the model should be stated clearly. The simulation emphasized control logic, sequencing, and subsystem coordination more than high-fidelity physical realism. This became especially clear in the treatment of part geometry and object handling. Simplified shapes improved stability, but they reduced realism. Likewise, repeatable Mobile Robot 2 performance required manual tuning of geometry, path, and grasp conditions that would likely need a more

formal engineering treatment in a production-ready system. These tradeoffs were reasonable for a proof-of-concept study focused on workflow validation, but they remain important limits on how broadly the present results can be generalized.

Another limitation concerns scalability. The current approach is well suited for basic sorting, staged transport, and controlled handoff tasks in an orderly environment. It is less ready for highly complex geometries, dense traffic interactions, multiple simultaneous loose objects, or more demanding real-time perception problems. As the manipulation and placement task became more geometry-sensitive, the amount of tuning and coordination effort increased. For that reason, the present model is best understood as a structured early-stage system model rather than a full warehouse digital twin. It demonstrates the architecture well, but further refinement would be needed for more complex operating conditions. The simulation also makes clear that centralized supervision brings both advantages and responsibilities. Because the supervisory layer owns the system's state and manages the major transitions, the process becomes easier to trace, debug, and validate. That is a major strength. At the same time, the quality of the full system depends heavily on the quality of that supervisory logic and on the correctness of the local interfaces beneath it. If the sequence logic is incomplete, if signal handling is inconsistent, or if local motion assumptions do not match the actual geometry of the station, the system can stall even when each individual subsystem appears to function on its own. The value of the present design is that it exposes those dependencies in simulation before hardware integration begins.

Overall, the evaluation is favorable. The project presents a coherent and realistic supervisory control concept for autonomous warehouse kitting and fulfillment. Its strongest features are the clarity of its centralized coordination, the practical use of PLC-style handshake logic, and the integration of the producer, conveyor, vision, mobile transport, and robotic handling stages into one controlled workflow. Its main limitations are the simplified physical model, the reduced realism associated with simplified geometry, and the amount of tuning required when tasks become more sensitive to manipulation and interface conditions. Those limits are acceptable for a proof-of-concept study. Viewed through the lens of real-world automation, the coordinator is best interpreted as a supervisory PLC layer, and that interpretation makes the project more practical, more defensible, and more consistent with how multi-asset systems are commonly controlled in industry.

7. Conclusion

This project presented a simulation-based design for an autonomous multi-robot warehouse kitting and fulfillment system. The study focused on a practical warehouse problem: how to coordinate material flow reliably when demand changes, and multiple robotic agents must work together across several dependent stages. Using CoppeliaSim, the project modeled the full fulfillment process from order entry through part generation, inspection, sorting, bin transport, and final unloading into shipping trays. The system used a producer object, a UR10 pick-and-place arm, a Pioneer3DX mobile base paired with a UR5 arm for Mobile 1, a KUKA youBot mobile manipulator for Mobile 2, and a UR10 arm for the Cobot. Together, these simulated assets showed how different robot types can be integrated into one coordinated warehouse workflow.

A central contribution of the project was the use of a coordinator-based control structure supported by local robot state machines and a shared request-busy-done signal protocol. This design provided a clear way to manage task sequencing, handoffs, bin movement, and order progress across the producer, pick-and-place robot, mobile robots, and Cobot. The coordinator functioned as a supervisory PLC layer by assigning work, monitoring status, confirming

completion, and updating order progress. This kept system-level decisions in one place while allowing each robot to focus on its local execution task. The project also demonstrated the value of simulation as a design and validation tool before physical implementation. The modeled environment made it possible to test robot motion, sensing, timing, signal logic, and selected disruption scenarios in a controlled setting. The simulation also revealed practical integration issues, including the need for consistent signal naming, reliable handshakes, standard pick/drop points, stable part geometry, and careful tuning of robot approach paths. These findings reinforced that successful multi-robot automation depends not only on individual robot capability, but also on dependable interfaces between robots, sensors, bins, and workflow states. At the same time, the project should be understood within its intended scope. The model emphasized coordination logic, task sequencing, and process validation rather than full physical realism. Simulation allowed fast iteration and simplified assumptions, but real systems must respect safety, tolerances, physical limits, payload limits, actuator capability, and hardware constraints. Future work could extend the model with more realistic motion routines, richer sensing uncertainty, stronger recovery logic, traffic and buffer effects, and broader performance testing under varied order conditions.

In conclusion, this project showed that a simulation-based multi-robot architecture can provide a practical and defensible framework for warehouse kitting and fulfillment. The developed system supports coordinated order execution, structured handoffs, and performance-oriented evaluation in a single modeled environment. It also provides a strong foundation for future refinement and offers a meaningful example of how autonomous robotic systems can be studied and validated before physical deployment.

8. Contributor Acknowledgments

The project team would like to acknowledge the contributions of all members of Group 7, HALJaMS Robotics. The team's coordinated efforts made this project possible. Aditya Abhang developed the pick-and-place robot workflow, including part pickup, inspection logic, and placement into the correct bin or scrap bin. Harshada Prasad Kulkarni developed the Mobile Robot 1 workflow for bin pickup and transport from the producer area to kitting. Lawrence Farhat developed the Mobile Robot 2 workflow for downstream transport to delivery preparation. Marx I. Bertrand led the development of the architecture and developed the Cobot workflow for part pickup from bins and placement into the correct shipping tray and empty bin return. Shlok Vijaykumar Ashtekar developed the parts producer workflow, including part generation, conveyor movement, and sensor-based stop logic. Juan Ernesto Rodriguez led overall system integration, helping define the architecture, control logic, signal-based communication, order-entry concept, and system-level validation approach. Together, the team contributed to an integrated multi-robot warehouse kitting simulation that combined robotic coordination, material flow, and performance-oriented system design.

9. Bibliography

- Arman, M. H., Aziz, R. A., Bari, A. B. M. M., Karmaker, C. L., & Dasgupta, S. (2025). Improving supply chain responsiveness through the drivers of Industry 5.0: Sustainability implications for emerging economies. *Sustainable Operations and Computers*, 6, 100503.
- Boysen, N., & de Koster, R. (2025). 50 years of warehousing research—An operations research perspective. *European Journal of Operational Research*, 320(3), 449–464.
<https://doi.org/10.1016/j.ejor.2024.03.026>
- Braunscheidel, M. J., & Suresh, N. C. (2009). The organizational antecedents of a firm's supply chain agility for risk mitigation and response. *Journal of Operations Management*, 27(2), 119-140.
- Dubey, V. K., & Veeramani, D. (2024). A decision-making framework for automating distribution centers in the retail supply. *Heliyon*, 10(10), e30854.
<https://doi.org/10.1016/j.heliyon.2024.e30854>
- Ho, G. T. S., Tang, V., Tong, P. H., & Tam, M. M. F. (2025). Demand-driven storage allocation for optimizing order picking processes. *Expert Systems with Applications*, 272, 126812.
<https://doi.org/10.1016/j.eswa.2025.126812>
- Keung, K. L., Lee, C. K. M., Xia, L., Liu, C., Liu, B., & Ji, P. (2023). A cyber-physical robotic mobile fulfillment system in smart manufacturing: The simulation aspect. *Robotics and Computer-Integrated Manufacturing*, 83, 102578.
<https://doi.org/10.1016/j.rcim.2023.102578>
- Leung, E. K. H. (2025). Total fulfillment management: Principles, practices and use cases. *Transportation Research Part E: Logistics and Transportation Review*, 194, 103888.
<https://doi.org/10.1016/j.tre.2024.103888>
- Massari, G. F., Nacchiero, R., & Giannoccaro, I. (2025). Transformative supply chains: The enabling role of digital technologies. *International Journal of Production Economics*, 283, 109562.
- Park, J., & Hong, S. (2025). Optimizing blocking and starving delays in sequential zone order picking systems through time-decomposed workload balancing. *Computers & Operations Research*, 180, 107060. <https://doi.org/10.1016/j.cor.2025.107060>
- Zhen, L., Tan, Z., de Koster, R., & Wang, S. (2023). How to deploy robotic mobile fulfillment systems. *Transportation Science*, 57(6), 1671-1695.
<https://doi.org/10.1287/trsc.2022.0265>

10. Appendix

Appendix A.

Table 1. Signal Map: Global / Order-level

Signal Name	Type	Write	Read	Notes
order_green	Int	Order Console	Coordinator	Requested qty (green)
order_blue	Int	Order Console	Coordinator	Requested qty (blue)
order_yellow	Int	Order Console	Coordinator	Requested qty (yellow)
order_red	Int	Order Console	Coordinator	Requested qty (red)
system_enabled	Int	Order Console / Coordinator	All robots / Coordinator	1 = system active
order_ready	Int	Order Console / Coordinator	Coordinator	1= ready to start
order_complete	Int	Coordinator	Order Console / UI	1 = order finished

Appendix B.

Table 2. Signal Map: Producer (Pick-and-Place, Producer Bins)

Signal Name	Type	Write	Read	Notes
req_pick_place	Int	Coordinator	Pick-and-Place	1 = request pick-place
pick_place_color	String	Coordinator	Pick-and-Place	Color to place
pick_place_bin_id	String	Coordinator	Pick-and-Place	Target producer bin id
pick_place_busy	Int	Pick-and-Place	Coordinator	1 – pick-place active
pick_place_done	Int	Pick-and-Place	Coordinator	1 = pick-place finished
last_picked_color	String	Coordinator	UI/Diagnostics	Last picked color
remain_red	Int	Coordinator	Coordinator	remaining red
remain_blue	Int	Coordinator	Coordinator	remaining blue
remain_yellow	Int	Coordinator	Coordinator	remaining yellow
remain_green	Int	Coordinator	Coordinator	remaining green

Appendix C.

Table 3. Signal Map: Kitting (Mobile 1 and Bins at Kitting)

Signal Name	Type	Write	Read	Notes
req_mobile1	Int	Coordinator	Mobile 1	1 = move bin producer → kitting
mobile1_bin_id	String	Coordinator	Mobile 1	Which producer bin to move
mobile1_busy	Int	Mobile 1	Coordinator	1 = mobile1 actively working
mobile1_done	Int	Mobile 1	Coordinator	1 = mobile1 finished move to kitting

Appendix D.

Table 4. Signal Map: Delivery (Mobile 2, Cobot, and Bins at Delivery)

Signal Name	Type	Write	Read	Notes
req_mobile2	Int	Coordinator	Mobile 2	1 = move bin from kitting → delivery
mobile2_bin_id	String	Coordinator	Mobile 2	which kitting bin to move
mobile2_busy	Int	Mobile 2	Coordinator	1 = mobile2 actively working
mobile2_done	Int	Mobile 2	Coordinator	1 = mobile2 finished move to delivery
req_cobot	Int	Coordinator	Cobot	1 = request cobot to unload a delivery bin
cobot_bin_id	String	Coordinator	Cobot	which delivery bin to unload
cobot_busy	Int	Cobot	Coordinator	1 = cobot actively unloading
cobot_done	Int	Cobot	Coordinator	1 = cobot finished unloading

Appendix E. Multi-Robot Integration Signal Guide

Purpose. This guide tells each teammate exactly which signals the coordinator sends, which signals each robot must send back, and which scene positions each robot should use when the assigned bin id is received. The coordinator is the only source of truth for bin assignment.

1. Fixed shared bin mapping

Table 5. Bin Mapping

Bin ID	Color	Upstream area	Next area	Used by
1	Yellow	Producer	Kitting → Delivery → Yellow tray	Pick-and-Place, Mobile 1, Mobile 2, Cobot
2	Blue	Producer	Kitting → Delivery → Blue tray	Pick-and-Place, Mobile 1, Mobile 2, Cobot
3	Green	Producer	Kitting → Delivery → Green tray	Pick-and-Place, Mobile 1, Mobile 2, Cobot
4	Red	Producer	Kitting → Delivery → Red tray	Pick-and-Place, Mobile 1, Mobile 2, Cobot

Important rule. A robot must not decide its own bin. It must read the assigned bin id from the coordinator, save it locally, execute only that bin task, then reply with busy/done acknowledgement.

2. Exact scene pick and drop locations

Use these exact object names from the scene/PDF when building local lookup tables.

Producer area

Table 6. Scene Pick and Drop Locations

Position name	Use
bin1_producer_pick	Producer pickup reference
bin1_producer_drop	Producer drop reference
bin2_producer_pick	Producer pickup reference
bin2_producer_drop	Producer drop reference
bin3_producer_pick	Producer pickup reference
bin3_producer_drop	Producer drop reference
bin4_producer_pick	Producer pickup reference
bin4_producer_drop	Producer drop reference

Kitting area

Table 7. Scene Pick and Drop Locations

Position name	Use
bin1_kitting_pick	Kitting pickup reference
bin1_kitting_drop	Kitting drop reference
bin2_kitting_pick	Kitting pickup reference
bin2_kitting_drop	Kitting drop reference
bin3_kitting_pick	Kitting pickup reference
bin3_kitting_drop	Kitting drop reference
bin4_kitting_pick	Kitting pickup reference
bin4_kitting_drop	Kitting drop reference

Delivery preparation area

Table 8. Scene Pick and Drop Locations

Position name	Use
bin1_delivery_pick	Delivery pickup reference
bin1_delivery_drop	Delivery drop reference
bin2_delivery_pick	Delivery pickup reference
bin2_delivery_drop	Delivery drop reference
bin3_delivery_pick	Delivery pickup reference
bin3_delivery_drop	Delivery drop reference
bin4_delivery_pick	Delivery pickup reference
bin4_delivery_drop	Delivery drop reference

Tray drop positions

Table 9. Scene Pick and Drop Locations

Position name	Use
tray_yellow_drop	Shipping tray drop reference
tray_blue_drop	Shipping tray drop reference
tray_green_drop	Shipping tray drop reference
tray_red_drop	Shipping tray drop reference

3. Standard coordinator handshake used by every robot

- Stay in IDLE until the coordinator request signal for that robot becomes 1 and the task data signal is valid.
- Read the assigned bin id once at task start and store it in a local variable.
- Set the robot busy signal to 1 as soon as the task is accepted.
- Execute only the assigned motion sequence for that exact bin id.
- Set the robot done signal to 1 when the motion sequence is complete.
- Wait in a WAIT_FOR_CLEAR state until the coordinator clears the request signal back to 0.
- Only after the request signal is cleared should the robot reset its own busy and done signals and return to IDLE.

4. Coordinator interface summary by robot

Table 10. Coordinator Interface Summary

Robot	Signals from coordinator	Signals back to coordinator	Task data	Robot must do	When to reset
Pick-and-Place	req_pick_place, reject_part	pick_place_busy, pick_place_done, detected_part_color	pick_place_bin_id	Detect the part color, wait for coordinator approval, then place into the assigned producer bin or reject path.	Do not clear busy/done until req_pick_place and any reject command used by the team are cleared.
Mobile Robot 1	req_mobile1	mobile1_busy, mobile1_done	mobile1_bin_id	Move the assigned producer bin from producer to kitting.	Do not clear busy/done until req_mobile1 is cleared.
Mobile Robot 2	req_mobile2	mobile2_busy, mobile2_done	mobile2_bin_id	Move the assigned kitting bin from kitting to delivery.	Do not clear busy/done until req_mobile2 is cleared.
Cobot	req_cobot	cobot_busy, cobot_done	cobot_bin_id	Unload the assigned delivery bin to the correct	Do not clear busy/done until

				shipping tray based on bin color.	req_cobot is cleared.
--	--	--	--	-----------------------------------	-----------------------

5. Robot-specific instructions

Pick-and-Place (Aditya)

- Wait for a part to be present at the producer pickup location.
- Classify the actual part color and write detected_part_color for the coordinator.
- Wait for the coordinator decision. If the part is accepted, the coordinator writes req_pick_place = 1 and pick_place_bin_id = 1..4. If the part is rejected, follow the reject path used by the team.
- Use a local lookup table keyed by pick_place_bin_id to select the exact producer destination.
- Place into the assigned producer drop position, then set pick_place_done = 1.

Table 11. Pick and Drop References

Bin ID	Color	Producer pick/drop references	Minimum local fields
1	Yellow	bin1 producer pick, bin1 producer drop	color, pick handle, drop handle, optional path
2	Blue	bin2 producer pick, bin2 producer drop	color, pick handle, drop handle, optional path
3	Green	bin3 producer pick, bin3 producer drop	color, pick handle, drop handle, optional path
4	Red	bin4 producer pick, bin4 producer drop	color, pick handle, drop handle, optional path

Mobile Robot 1 (Harshada)

- Wait in IDLE until req_mobile1 = 1.
- Read mobile1_bin_id once and store it locally.
- Set mobile1_busy = 1, move the assigned bin from producer to kitting, then set mobile1_done = 1.
- Wait until the coordinator clears req_mobile1 back to 0 before resetting local status.

Table 12. Pick and Drop References

Bin ID	Color	Producer pickup → kitting drop	Minimum local fields
1	Yellow	bin1 producer pick → bin1 kitting drop	pick handle, drop handle, optional path
2	Blue	bin2 producer pick → bin2 kitting drop	pick handle, drop handle, optional path
3	Green	bin3 producer pick → bin3 kitting drop	pick handle, drop handle, optional path
4	Red	bin4 producer pick → bin4 kitting drop	pick handle, drop handle, optional path

Mobile Robot 2 (Lawrence)

- Wait in IDLE until req_mobile2 = 1.
- Read mobile2_bin_id once and store it locally.
- Set mobile2_busy = 1, move the assigned bin from kitting to delivery, then set mobile2_done = 1.
- Wait until the coordinator clears req_mobile2 back to 0 before resetting local status.

Table 13. Pick and Drop References

Bin ID	Color	Kitting pickup → delivery drop	Minimum local fields
1	Yellow	bin1 kitting pick → bin1 delivery drop	pick handle, drop handle, optional path
2	Blue	bin2 kitting pick → bin2 delivery drop	pick handle, drop handle, optional path
3	Green	bin3 kitting pick → bin3 delivery drop	pick handle, drop handle, optional path
4	Red	bin4 kitting pick → bin4 delivery drop	pick handle, drop handle, optional path

Cobot (Marx)

Wait in IDLE until req_cobot = 1.

Read cobot_bin_id once and store it locally.

Set `cobot_busy = 1`, unload only the assigned delivery bin, place into the correct tray, then set `cobot_done = 1`.

Do not change `remain_*` in the cobot script. The coordinator updates completion and remaining quantities after `cobot_done`.

Wait until the coordinator clears `req_cobot` back to 0 before resetting local status.

Table 14. Pick and Drop References

Bin ID	Color	Delivery pickup → tray drop	Minimum local fields
1	Yellow	<code>bin1_delivery_pick → tray_yellow_drop</code>	<code>pick handle, tray handle, color, optional path</code>
2	Blue	<code>bin2_delivery_pick → tray_blue_drop</code>	<code>pick handle, tray handle, color, optional path</code>
3	Green	<code>bin3_delivery_pick → tray_green_drop</code>	<code>pick handle, tray handle, color, optional path</code>
4	Red	<code>bin4_delivery_pick → tray_red_drop</code>	<code>pick handle, tray handle, color, optional path</code>

6. Ownership rules that should not change

- Only the coordinator assigns bin ids.
- Only the coordinator clears `req_*` request signals.
- Only the coordinator updates `remain_*` and overall order completion.
- Each robot owns only its own busy and done signals.
- Each robot should print a clear diagnostic and stay safe if the assigned bin id is blank, invalid, or missing from the local lookup table.

7. Robot-by-robot coordination templates

Pick-and-Place (Aditya)

Coordinator interface and required local implementation details for this robot.

Signals from coordinator	<code>part_ready_at_pick</code> (or stable producer-ready condition), <code>req_pick_place</code> , <code>reject_part</code> , <code>pick_place_bin_id</code> The coordinator is the only source of truth for the accepted destination bin id.
Signals back to coordinator	<code>detected_part_color</code> as soon as color is classified <code>pick_place_busy = 1</code> when the task is accepted <code>pick_place_done = 1</code> after place or reject handling is complete
Task data to store locally at task start	<code>detected_part_color</code> <code>pick_place_bin_id</code>
Exact scene positions to support	<code>bin1_producer_pick</code> , <code>bin1_producer_drop</code> <code>bin2_producer_pick</code> , <code>bin2_producer_drop</code> <code>bin3_producer_pick</code> , <code>bin3_producer_drop</code> <code>bin4_producer_pick</code> , <code>bin4_producer_drop</code>
Robot must do	Wait for part at producer pickup location. Inspect and classify the actual part color. Report <code>detected_part_color</code> . Wait for coordinator approval through <code>req_pick_place</code> and <code>pick_place_bin_id</code> , or <code>reject_part</code> for reject routing. Pick the part, move to the assigned producer drop location, place the part, and hold done until cleared.

Logic Structure (replace the red text with your robot specific code)

```
function sysCall_init()
    sim = require('sim')

    state = 'WAIT_FOR_PART'

    pickPlaceTable = {
        [1] = {color='yellow', producerPickName='bin1_producer_pick', producerDropName='bin1_producer_drop'},
        [2] = {color='blue',  producerPickName='bin2_producer_pick', producerDropName='bin2_producer_drop'},
        [3] = {color='green',  producerPickName='bin3_producer_pick', producerDropName='bin3_producer_drop'},
        [4] = {color='red',    producerPickName='bin4_producer_pick', producerDropName='bin4_producer_drop'}
    }

    detectedColor = ""
    activeTask = nil
    producerPickHandle = nil
    producerDropHandle = nil
end

local function getInt(name)
    return sim.getInt32Signal(name) or 0
end

local function setInt(name, value)
    sim.setInt32Signal(name, value)
end

local function getStr(name)
    return sim.getStringSignal(name) or ""
end

local function setStr(name, value)
    sim.setStringSignal(name, value or "")
end

function sysCall_actuation()
    local partReady = getInt('part_ready_at_pick')
    local reqPickPlace = getInt('req_pick_place')
    local rejectPart = getInt('reject_part')
    local binIdStr = getStr('pick_place_bin_id')

    if state == 'WAIT_FOR_PART' then
        if partReady == 1 then
            -----
            -- ADITYA: INSPECTION LOGIC
            -----

            local robotTarget = sim.getObject('/PickPlaceTarget')

            -- Move near the incoming part pickup or inspection point
            -- Replace this target name if your scene uses a different one
            local incomingPartHandle = sim.getObject('/bin1_producer_pick')
            local incomingPartPose = sim.getObjectPose(incomingPartHandle, -1)
            sim.setObjectPose(robotTarget, -1, incomingPartPose)

            -- Detect actual color of the part
            detectedColor = 'yellow' -- example only
            setStr('detected_part_color', detectedColor)

            -----
        end
    end
end
```



```

-- END ADITYA SECTION
-----

state = 'WAIT_FOR_DECISION'
end
end

if state == 'WAIT_FOR_DECISION' then
    if reqPickPlace == 1 and binIdStr ~= " " then
        local assignedId = tonumber(binIdStr)
        local task = pickPlaceTable[assignedId]

        if task == nil then
            print('PickPlace: invalid assigned bin ID')
            return
        end

        activeTask = task
        producerPickHandle = sim.getObject('/' .. activeTask.producerPickName)
        producerDropHandle = sim.getObject('/' .. activeTask.producerDropName)

        setInt('pick_place_busy', 1)
        state = 'RUN_ACCEPTED_TASK'
    elseif rejectPart == 1 then
        setInt('pick_place_busy', 1)
        state = 'RUN_REJECT_TASK'
    end
end

if state == 'RUN_ACCEPTED_TASK' then
    -----
    -- ADITYA: ACCEPTED PATH MOTION LOGIC
    -----

    local robotTarget = sim.getObject('/PickPlaceTarget')

    local producerPickPose = sim.getObjectPose(producerPickHandle, -1)
    sim.setObjectPose(robotTarget, -1, producerPickPose)

    -- Add logic to move the robot to the current pickup position
    -- and complete the local pickup routine for the part.

    local producerDropPose = sim.getObjectPose(producerDropHandle, -1)
    sim.setObjectPose(robotTarget, -1, producerDropPose)

    -- Add logic to move the robot to the assigned producer drop position
    -- and complete the local release routine so the part is placed in the correct producer bin.

    -----
    -- END ADITYA SECTION
    -----

    setInt('pick_place_done', 1)
    state = 'WAIT_FOR_CLEAR'
end

if state == 'RUN_REJECT_TASK' then
    -----
    -- ADITYA: REJECT PATH MOTION LOGIC
    -----

```

```

-----
local robotTarget = sim.getObject('/PickPlaceTarget')

-- Replace this with the actual reject location object in the scene
local rejectHandle = sim.getObject('/reject_drop')
local rejectPose = sim.getObjectPose(rejectHandle, -1)
sim.setObjectPose(robotTarget, -1, rejectPose)

-- Add logic to move the robot to the reject location
-- and complete the local release or discard routine for the rejected part.

-----
-- END ADITYA SECTION
-----

setInt('pick place done', 1)
state = 'WAIT_FOR_CLEAR'
end

if state == 'WAIT_FOR_CLEAR' then
  if reqPickPlace == 0 and rejectPart == 0 then
    setInt('pick_place_busy', 0)
    setInt('pick_place_done', 0)

    detectedColor = "
    activeTask = nil
    producerPickHandle = nil
    producerDropHandle = nil

    state = 'WAIT_FOR_PART'
  end
end
end
end

```

Mobile Robot 1(Harshada)

Coordinator interface and required local implementation details for this robot.

Signals from coordinator	req_mobile1, mobile1_bin_id
Signals back to coordinator	mobile1_busy = 1 when the task is accepted mobile1_done = 1 after the producer-to-kitting move is complete
Task data to store locally at task start	mobile1_bin_id
Exact scene positions to support	bin1_producer_pick -> bin1_kitting_drop bin2_producer_pick -> bin2_kitting_drop bin3_producer_pick -> bin3_kitting_drop bin4_producer_pick -> bin4_kitting_drop
Robot must do	Stay in IDLE until req_mobile1 = 1. Read mobile1_bin_id once and store it locally. Use only the assigned bin id. Do not search for a color or choose a different bin. Pick the assigned producer bin and place it at the matching kitting drop

	location. Set mobile1_done = 1 and wait for req_mobile1 to clear before resetting.
--	---

Logic Structure (replace the red text with your robot specific code)

```

function sysCall_init()
    sim = require('sim')

    state = 'IDLE'

    mobile1Table = {
        [1] = {color='yellow', producerPickName='bin1_producer_pick', kittingDropName='bin1_kitting_drop'},
        [2] = {color='blue', producerPickName='bin2_producer_pick', kittingDropName='bin2_kitting_drop'},
        [3] = {color='green', producerPickName='bin3_producer_pick', kittingDropName='bin3_kitting_drop'},
        [4] = {color='red', producerPickName='bin4_producer_pick', kittingDropName='bin4_kitting_drop'}
    }

    activeTask = nil
    producerPickHandle = nil
    kittingDropHandle = nil
end

local function getInt(name)
    return sim.getInt32Signal(name) or 0
end

local function setInt(name, value)
    sim.setInt32Signal(name, value)
end

local function getStr(name)
    return sim.getStringSignal(name) or ""
end

function sysCall_actuation()
    local reqMobile1 = getInt('req_mobile1')
    local binIdStr = getStr('mobile1_bin_id')

    if state == 'IDLE' then
        if reqMobile1 == 1 and binIdStr ~= "" then
            local assignedId = tonumber(binIdStr)
            local task = mobile1Table[assignedId]

            if task == nil then
                print('Mobile1: invalid assigned bin ID')
                return
            end

            activeTask = task
        end
    end
end

```

```

    producerPickHandle = sim.getObject('/') .. activeTask.producerPickName)
    kittingDropHandle = sim.getObject('/') .. activeTask.kittingDropName)

    setInt('mobile1_busy', 1)
    state = 'RUN_TASK'
end
end

if state == 'RUN_TASK' then
    -----
    -- HARSHADA: MOTION LOGIC
    -----

    local robotTarget = sim.getObject('/Mobile1Target')
    local producerPickPose = sim.getObjectPose(producerPickHandle, -1)
    sim.setObjectPose(robotTarget, -1, producerPickPose)

    -- Add logic to move the robot to the assigned producer pickup position
    -- and complete the local pickup routine for that assigned bin.

    local kittingDropPose = sim.getObjectPose(kittingDropHandle, -1)
    sim.setObjectPose(robotTarget, -1, kittingDropPose)

    -- Add logic to move the robot to the matching kitting drop position
    -- and complete the local release routine so the bin is left at kitting.

    -----
    -- END HARSHADA SECTION
    -----

    setInt('mobile1_done', 1)
    state = 'WAIT_FOR_CLEAR'
end

if state == 'WAIT_FOR_CLEAR' then
    if reqMobile1 == 0 then
        setInt('mobile1_busy', 0)
        setInt('mobile1_done', 0)

        activeTask = nil
        producerPickHandle = nil
        kittingDropHandle = nil

        state = 'IDLE'
    end
end
end
end

```

Mobile Robot 2 (Lawrence)

Coordinator interface and required local implementation details for this robot.

Signals from coordinator	req_mobile2, mobile2_bin_id
Signals back to coordinator	mobile2_busy = 1 when the task is accepted mobile2_done = 1 after the kitting-to-delivery move is complete
Task data to store locally at task start	mobile2_bin_id
Exact scene positions to support	bin1_kitting_pick -> bin1_delivery_drop bin2_kitting_pick -> bin2_delivery_drop bin3_kitting_pick -> bin3_delivery_drop bin4_kitting_pick -> bin4_delivery_drop
Robot must do	Stay in IDLE until req_mobile2 = 1. Read mobile2_bin_id once and store it locally. Use only the assigned bin id from the coordinator. Pick the assigned kitting bin and place it at the matching delivery drop location. Set mobile2_done = 1 and wait for req_mobile2 to clear before resetting.

Logic Structure (replace the red text with your robot specific code)

```
function sysCall_init()
    sim = require('sim')

    state = 'IDLE'

    mobile2Table = {
        [1] = {color='yellow', kittingPickName='bin1_kitting_pick', deliveryDropName='bin1_delivery_drop'},
        [2] = {color='blue', kittingPickName='bin2_kitting_pick', deliveryDropName='bin2_delivery_drop'},
        [3] = {color='green', kittingPickName='bin3_kitting_pick', deliveryDropName='bin3_delivery_drop'},
        [4] = {color='red', kittingPickName='bin4_kitting_pick', deliveryDropName='bin4_delivery_drop'}
    }

    activeTask = nil
    kittingPickHandle = nil
    deliveryDropHandle = nil
end

local function getInt(name)
    return sim.getInt32Signal(name) or 0
end

local function setInt(name, value)
    sim.setInt32Signal(name, value)
end

local function getStr(name)
```

```

    return sim.getStringSignal(name) or "
end

function sysCall_actuation()
    local reqMobile2 = getInt('req_mobile2')
    local binIdStr = getStr('mobile2_bin_id')

    if state == 'IDLE' then
        if reqMobile2 == 1 and binIdStr ~= "" then
            local assignedId = tonumber(binIdStr)
            local task = mobile2Table[assignedId]

            if task == nil then
                print('Mobile2: invalid assigned bin ID')
                return
            end

            activeTask = task
            kittingPickHandle = sim.getObject('/') .. activeTask.kittingPickName)
            deliveryDropHandle = sim.getObject('/') .. activeTask.deliveryDropName)

            setInt('mobile2_busy', 1)
            state = 'RUN_TASK'
        end
    end

    if state == 'RUN_TASK' then
        -----
        -- LAWRENCE: MOTION LOGIC
        -----

        local robotTarget = sim.getObject('/Mobile2Target')

        local kittingPickPose = sim.getObjectPose(kittingPickHandle, -1)
        sim.setObjectPose(robotTarget, -1, kittingPickPose)

        -- Add logic to move the robot to the assigned kitting pickup position
        -- and complete the local pickup routine for that assigned bin.

        local deliveryDropPose = sim.getObjectPose(deliveryDropHandle, -1)
        sim.setObjectPose(robotTarget, -1, deliveryDropPose)

        -- Add logic to move the robot to the matching delivery drop position
        -- and complete the local release routine so the bin is left at delivery.

        -----
    end
end

```

```

-- END LAWRENCE SECTION
-----

setInt('mobile2_done', 1)
state = 'WAIT_FOR_CLEAR'
end

if state == 'WAIT_FOR_CLEAR' then
  if reqMobile2 == 0 then
    setInt('mobile2_busy', 0)
    setInt('mobile2_done', 0)

    activeTask = nil
    kittingPickHandle = nil
    deliveryDropHandle = nil

    state = 'IDLE'
  end
end
end
end

```

Cobot (Marx)

Coordinator interface and required local implementation details for this robot.

Signals from coordinator	req_cobot, cobot_bin_id
Signals back to coordinator	cobot_busy = 1 when the task is accepted cobot_done = 1 after unload is complete
Task data to store locally at task start	cobot_bin_id
Exact scene positions to support	bin1_delivery_pick -> tray_yellow_drop bin2_delivery_pick -> tray_blue_drop bin3_delivery_pick -> tray_green_drop bin4_delivery_pick -> tray_red_drop
Robot must do	Stay in IDLE until req_cobot = 1. Read cobot_bin_id once and store it locally. Use the assigned bin id to select the delivery pickup and tray drop. Do not change remain_* in the robot script. Set cobot_done = 1 and wait for req_cobot to clear.

Logic Structure (replace the red text with your robot specific code)

```

function sysCall_init()
  sim = require('sim')

  state = 'IDLE'

  cobotTable = {

```

```

[1] = {color='yellow', deliveryPickName='bin1_delivery_pick', trayDropName='tray_yellow_drop'},
[2] = {color='blue', deliveryPickName='bin2_delivery_pick', trayDropName='tray_blue_drop'},
[3] = {color='green', deliveryPickName='bin3_delivery_pick', trayDropName='tray_green_drop'},
[4] = {color='red', deliveryPickName='bin4_delivery_pick', trayDropName='tray_red_drop'}
}

activeTask = nil
deliveryPickHandle = nil
trayDropHandle = nil
end

local function getInt(name)
    return sim.getInt32Signal(name) or 0
end

local function setInt(name, value)
    sim.setInt32Signal(name, value)
end

local function getStr(name)
    return sim.getStringSignal(name) or ""
end

function sysCall_actuation()
    local reqCobot = getInt('req_cobot')
    local binIdStr = getStr('cobot_bin_id')

    if state == 'IDLE' then
        if reqCobot == 1 and binIdStr ~= "" then
            local assignedId = tonumber(binIdStr)
            local task = cobotTable[assignedId]

            if task == nil then
                print('Cobot: invalid assigned bin ID')
                return
            end

            activeTask = task
            deliveryPickHandle = sim.getObject('/') .. activeTask.deliveryPickName
            trayDropHandle = sim.getObject('/') .. activeTask.trayDropName

            setInt('cobot_busy', 1)
            state = 'RUN_TASK'
        end
    end

    if state == 'RUN_TASK' then
        -----
        -- MARX: MOTION LOGIC
    end
end

```



```

-----

local robotTarget = sim.getObject('/CobotTarget')
local deliveryPickPose = sim.getObjectPose(deliveryPickHandle, -1)
sim.setObjectPose(robotTarget, -1, deliveryPickPose)

-- Add logic to move the robot to the assigned delivery pickup position
-- and complete the local pickup or unload routine for that assigned bin.

local trayDropPose = sim.getObjectPose(trayDropHandle, -1)
sim.setObjectPose(robotTarget, -1, trayDropPose)

-- Add logic to move the robot to the matching tray drop position
-- and complete the local release routine so the part is placed in the correct tray.

-----

-- END MARX SECTION
-----

setInt('cobot_done', 1)
state = 'WAIT_FOR_CLEAR'
end

if state == 'WAIT_FOR_CLEAR' then
  if reqCobot == 0 then
    setInt('cobot_busy', 0)
    setInt('cobot_done', 0)

    activeTask = nil
    deliveryPickHandle = nil
    trayDropHandle = nil

    state = 'IDLE'
  end
end
end
end

```

8. Bin Map

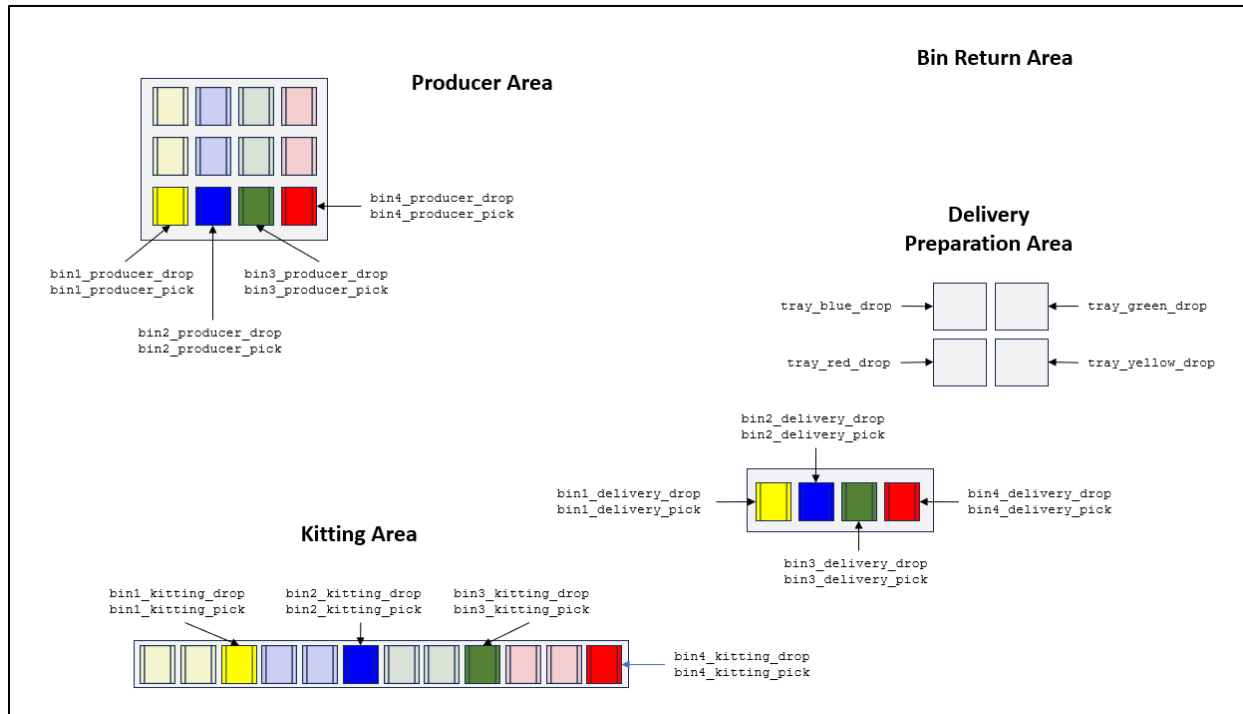


Figure 3. Bin Map

9. Testing

Independent Robot Testing Procedure

Each teammate should test their robot independently before full-system integration. The purpose of independent testing is to confirm that the robot can resolve its assigned scene positions, execute the correct motion for a single assigned bin ID, and return the expected busy/done response without relying on the Coordinator, Order Console, or the other robots.

General testing principle

For independent testing, the robot should behave exactly as it would under normal coordinator control, except that the request and task data are provided locally for test purposes. The recommended approach is to keep the same request-busy-done structure and add a small local test mode that injects the request signal and assigned bin ID when the simulation starts.

Each teammate should verify four things during standalone testing:

1. the local lookup table resolves the correct scene objects or poses for the assigned bin ID
2. the robot motion goes to the correct pickup location
3. the robot motion goes to the correct drop location
4. the robot sets its busy and done signals in the correct order

Pick-and-Place standalone test

The Pick-and-Place robot should be tested in two ways.

First, test the vision or inspection portion by confirming that the robot can:

- move to the inspection or producer pickup pose
- detect or classify the part color
- write detected_part_color

Second, test the coordinator-approved placement portion by manually forcing:

```
req_pick_place = 1
pick_place_bin_id = '1', '2', '3', or '4'
```

The teammate should then verify that the robot:

- reads pick_place_bin_id
- resolves the correct producer destination from pickPlaceTable
- sets pick_place_busy = 1
- moves to the correct assigned producer drop location
- sets pick_place_done = 1

Recommended standalone test cases:

- pick_place_bin_id = '1' should route to bin1_producer_drop
- pick_place_bin_id = '2' should route to bin2_producer_drop
- pick_place_bin_id = '3' should route to bin3_producer_drop
- pick_place_bin_id = '4' should route to bin4_producer_drop

Mobile Robot 1 standalone test

For Mobile Robot 1, the teammate should force:

```
system_enabled = 1
req_mobile1 = 1
mobile1_bin_id = '1', '2', '3', or '4'
```

The robot should then:

- read mobile1_bin_id
- resolve the assigned producer pickup and kitting drop positions from mobile1Table
- set mobile1_busy = 1
- move only the assigned bin route
- set mobile1_done = 1

Required motion checks:

- bin 1: bin1_producer_pick → bin1_kitting_drop
- bin 2: bin2_producer_pick → bin2_kitting_drop
- bin 3: bin3_producer_pick → bin3_kitting_drop
- bin 4: bin4_producer_pick → bin4_kitting_drop

Mobile Robot 2 standalone test

For Mobile Robot 2, the teammate should force:

```
system_enabled = 1
req_mobile2 = 1
mobile2_bin_id = '1', '2', '3', or '4'
```

The robot should then:

- read mobile2_bin_id

- resolve the assigned kitting pickup and delivery drop positions from `mobile2Table`
- `set mobile2_busy = 1`
- move only the assigned bin route
- `set mobile2_done = 1`

Required motion checks:

- bin 1: `bin1_kitting_pick` → `bin1_delivery_drop`
- bin 2: `bin2_kitting_pick` → `bin2_delivery_drop`
- bin 3: `bin3_kitting_pick` → `bin3_delivery_drop`
- bin 4: `bin4_kitting_pick` → `bin4_delivery_drop`

Cobot standalone test

For the Cobot, the teammate should force:

```
system_enabled = 1
req_cobot = 1
cobot_bin_id = '1', '2', '3', or '4'
```

The robot should then:

- `read cobot_bin_id`
- resolve the assigned delivery pickup and tray drop positions from `cobotTable`
- `set cobot_busy = 1`
- execute only the assigned unload path
- `set cobot_done = 1`

Required motion checks:

- bin 1: `bin1_delivery_pick` → `tray_yellow_drop`
- bin 2: `bin2_delivery_pick` → `tray_blue_drop`
- bin 3: `bin3_delivery_pick` → `tray_green_drop`
- bin 4: `bin4_delivery_pick` → `tray_red_drop`

Recommended independent test sequence

Each teammate should run the following sequence:

1. test one bin ID only and verify pickup and drop positions
2. test all four bin IDs one at a time
3. verify that invalid or blank bin IDs are rejected safely
4. verify that busy becomes 1 at task start
5. verify that done becomes 1 only after motion completes
6. verify that the robot returns to IDLE correctly after the request is cleared

Independent test safety rules

During standalone testing:

- disable the Coordinator script unless specifically testing coordinator interaction
- disable the Order Console unless it is needed for the test
- disable unrelated robot scripts if they can interfere with shared signals
- use the exact same local lookup table that will be used in integrated operation

- do not change the shared bin mapping during testing

Fixed bin mapping used during all independent tests

The same shared bin map must be used during all standalone tests:

- bin1 = yellow
- bin2 = blue
- bin3 = green
- bin4 = red

Example standalone test notes for script templates

A teammate may add a short test block such as:

```
-- Standalone test mode
local TEST_MODE = true
local TEST_BIN_ID = '1'
```

Then, when test mode is enabled, the script can inject the request and assigned bin ID locally before running the normal task logic.

Expected result of independent testing

At the end of independent testing, each teammate should be able to confirm that:

- the robot uses only the assigned bin ID
- the robot does not guess the next task
- the robot reaches the correct scene positions for that assigned bin
- the robot returns the correct busy/done acknowledgement sequence
- the robot is ready to connect to the Coordinator without changing the core motion flow

10. Coordinator Script (for Reference)

```
function sysCall_init()
  sim = require('sim')

  BIN_CAPACITY = 1
  COLORS = {'yellow', 'blue', 'green', 'red'}

  initBins()
  resetCoordinatorRuntime(true)
end

-- =====
-- BIN MODEL
-- =====

function initBins()
  bins = {
    [1] = {id = 1, color = 'yellow', qty = 0, state = 'empty', location = 'producer', reserved = false},
    [2] = {id = 2, color = 'blue', qty = 0, state = 'empty', location = 'producer', reserved = false},
    [3] = {id = 3, color = 'green', qty = 0, state = 'empty', location = 'producer', reserved = false},
    [4] = {id = 4, color = 'red', qty = 0, state = 'empty', location = 'producer', reserved = false}
  }
end

-- =====
```

```

-- SIGNAL HELPERS
-- =====
local function getInt(name)
    return sim.getInt32Signal(name) or 0
end

local function setInt(name, value)
    sim.setInt32Signal(name, value)
end

local function getStr(name)
    return sim.getStringSignal(name) or ""
end

local function setStr(name, value)
    sim.setStringSignal(name, value or "")
end

local function clearStr(name)
    sim.clearStringSignal(name)
end

-- =====
-- RESET / RUNTIME STATE
-- =====
function resetCoordinatorSignals()
    local intSignals = {
        'req_pick_place', 'pick_place_done',
        'req_mobile1', 'mobile1_done',
        'req_mobile2', 'mobile2_done',
        'req_cobot', 'cobot_done',

        'order_complete',
        'pick_place_busy', 'mobile1_busy', 'mobile2_busy', 'cobot_busy',

        'remain_red', 'remain_blue', 'remain_yellow', 'remain_green',

        'reject_part'
    }

    for _, s in ipairs(intSignals) do
        setInt(s, 0)
    end

    local strSignals = {
        'pick_place_bin_id',
        'mobile1_bin_id',
        'mobile2_bin_id',
        'cobot_bin_id',
        'last_picked_color'
    }

    for _, s in ipairs(strSignals) do
        setStr(s, "")
    end
end

```

```

function resetCoordinatorRuntime(fullReset)
    activeOrder = false
    lastSystemEnabled = getInt('system_enabled')

    resetCoordinatorSignals()

    if fullReset then
        initBins()
    else
        for _, bin in pairs(bins) do
            bin.qty = 0
            bin.state = 'empty'
            bin.location = 'producer'
            bin.reserved = false
        end
    end
end

-- =====
-- ORDER HELPERS
-- =====

local function remainSignal(color)
    return 'remain_' .. color
end

local function orderSignal(color)
    return 'order_' .. color
end

local function copyOrderToRemain()
    for _, color in ipairs(COLORS) do
        setInt(remainSignal(color), getInt(orderSignal(color)))
    end
end

local function totalRemain()
    local total = 0
    for _, color in ipairs(COLORS) do
        total = total + getInt(remainSignal(color))
    end
    return total
end

local function allRemainZero()
    return totalRemain() == 0
end

local function anyRobotBusyOrRequested()
    return getInt('req_pick_place') == 1
        or getInt('req_mobile1') == 1
        or getInt('req_mobile2') == 1
        or getInt('req_cobot') == 1
        or getInt('reject_part') == 1
        or getInt('pick_place_busy') == 1
        or getInt('mobile1_busy') == 1

```

```

    or getInt('mobile2_busy') == 1
    or getInt('cobot_busy') == 1
end

function startOrderIfNeeded()
    local enabled = getInt('system_enabled')
    local ready = getInt('order_ready')

    if enabled == 0 or ready == 0 or activeOrder then
        return
    end

    copyOrderToRemain()
    setInt('order_complete', 0)
    activeOrder = true

    print('Coordinator: order started')
end

-- =====
-- BIN LOOKUP HELPERS
-- =====

local function getBinById(binId)
    return bins[binId]
end

local function findProducerFillBin(color)
    for _, bin in pairs(bins) do
        if bin.color == color
            and bin.location == 'producer'
            and (bin.state == 'empty' or bin.state == 'filling')
            and bin.qty < BIN_CAPACITY
            and not bin.reserved then
            return bin
        end
    end
    return nil
end

local function findReadyProducerBin()
    for _, bin in pairs(bins) do
        if bin.location == 'producer'
            and bin.state == 'ready_at_producer'
            and not bin.reserved then
            return bin
        end
    end
    return nil
end

local function findNeededKittingBin()
    for _, color in ipairs(COLORS) do
        if getInt(remainSignal(color)) > 0 then
            for _, bin in pairs(bins) do
                if bin.color == color
                    and bin.location == 'kitting'

```



```

        and bin.state == 'at_kitting'
        and not bin.reserved
        and bin.qty > 0 then
            return bin
        end
    end
end
end
return nil
end

local function findDeliveryBin()
    for _, bin in pairs(bins) do
        if bin.location == 'delivery'
            and bin.state == 'at_delivery'
            and not bin.reserved
            and bin.qty > 0 then
                return bin
            end
        end
    end
    return nil
end

local function targetQtyForBin(color, currentQty)
    local remain = getInt(remainSignal(color))
    if remain <= 0 then
        return 0
    end
    return math.min(BIN_CAPACITY, remain)
end

-- =====
-- PICK-AND-PLACE: VISION-BASED ASSIGN / REJECT
-- =====

local function canIssuePickPlaceDecision()
    return getInt('req_pick_place') == 0
        and getInt('reject_part') == 0
        and getInt('pick_place_busy') == 0
end

function maybeRequestPickPlace()
    if not canIssuePickPlaceDecision() then
        return
    end

    -- Wait until a part is present and the Pick-and-Place robot has classified it
    if getInt('part_ready_at_pick') == 0 then
        return
    end

    local detectedColor = getStr('detected_part_color')
    if detectedColor == "" then
        return
    end

    local remain = getInt(remainSignal(detectedColor))

```

```

local bin = findProducerFillBin(detectedColor)

-- Reject if color is unknown, not needed, or no legal fill bin is available
if remain <= 0 or not bin then
    setInt('reject_part', 1)
    print(string.format('Coordinator: rejecting detected %s part', detectedColor))
    return
end

-- Accept the part and assign the exact producer bin
bin.state = 'filling'
setStr('pick_place_bin_id', tostring(bin.id))
setInt('req_pick_place', 1)

print(string.format(
    'Coordinator: accepted detected %s part, assigned producer bin %d',
    detectedColor, bin.id
))
end

function handlePickPlaceDone()
    if getInt('pick_place_done') == 0 then
        return
    end

    -- Accepted placement path
    if getInt('req_pick_place') == 1 then
        local detectedColor = getStr('detected_part_color')
        local binId = tonumber(getStr('pick_place_bin_id'))
        local bin = getBinById(binId)

        if bin and detectedColor ~= " and bin.color == detectedColor then
            bin.qty = math.min(BIN_CAPACITY, bin.qty + 1)

            local targetQty = targetQtyForBin(bin.color, bin.qty)
            if bin.qty >= targetQty and targetQty > 0 then
                bin.state = 'ready_at_producer'
            else
                bin.state = 'filling'
            end

            setStr('last_picked_color', detectedColor)

            print(string.format(
                'Coordinator: placed %s part into bin %d, qty=%d',
                detectedColor, bin.id, bin.qty
            ))
        else
            print('Coordinator: Pick-and-Place done received, but assignment/color validation failed')
        end

        setInt('req_pick_place', 0)
        setStr('pick_place_bin_id', "")
    end

    -- Reject path

```

```

if getInt('reject_part') == 1 then
    local detectedColor = getStr('detected_part_color')
    print(string.format(
        'Coordinator: reject path completed for detected color %s',
        detectedColor ~= " " and detectedColor or 'unknown'
    ))
    setInt('reject_part', 0)
end

-- Common cleanup after either accept or reject
setInt('pick_place_done', 0)
clearStr('detected_part_color')
setInt('part_ready_at_pick', 0)
end

-- =====
-- MOBILE 1: PRODUCER -> KITTING
-- =====

function maybeRequestMobile1()
    if getInt('req_mobile1') == 1 or getInt('mobile1_busy') == 1 then
        return
    end

    local bin = findReadyProducerBin()
    if not bin then
        return
    end

    bin.reserved = true
    bin.state = 'moving_to_kitting'
    setStr('mobile1_bin_id', tostring(bin.id))
    setInt('req_mobile1', 1)

    print(string.format('Coordinator: requested Mobile 1 for bin %d', bin.id))
end

function handleMobile1Done()
    if getInt('mobile1_done') == 0 then
        return
    end

    local binIdStr = getStr('mobile1_bin_id')
    local binId = tonumber(binIdStr)
    local bin = getBinById(binId)

    if bin then
        bin.location = 'kitting'
        bin.state = 'at_kitting'
        bin.reserved = false
        print(string.format('Coordinator: bin %d arrived at kitting', bin.id))
    else
        print('Coordinator: Mobile 1 done received, but mobile1_bin_id was invalid')
    end

    setInt('req_mobile1', 0)
    setInt('mobile1_done', 0)
end

```

```

    setStr('mobile1_bin_id', "")
end

-- =====
-- MOBILE 2: KITTING -> DELIVERY
-- =====

function maybeRequestMobile2()
    if getInt('req_mobile2') == 1 or getInt('mobile2_busy') == 1 then
        return
    end

    local bin = findNeededKittingBin()
    if not bin then
        return
    end

    bin.reserved = true
    bin.state = 'moving_to_delivery'
    setStr('mobile2_bin_id', tostring(bin.id))
    setInt('req_mobile2', 1)

    print(string.format('Coordinator: requested Mobile 2 for bin %d', bin.id))
end

function handleMobile2Done()
    if getInt('mobile2_done') == 0 then
        return
    end

    local binIdStr = getStr('mobile2_bin_id')
    local binId = tonumber(binIdStr)
    local bin = getBinById(binId)

    if bin then
        bin.location = 'delivery'
        bin.state = 'at_delivery'
        bin.reserved = false
        print(string.format('Coordinator: bin %d arrived at delivery', bin.id))
    else
        print('Coordinator: Mobile 2 done received, but mobile2_bin_id was invalid')
    end

    setInt('req_mobile2', 0)
    setInt('mobile2_done', 0)
    setStr('mobile2_bin_id', "")
end

-- =====
-- COBOT: DELIVERY -> TRAY
-- =====

function maybeRequestCobot()
    if getInt('req_cobot') == 1 or getInt('cobot_busy') == 1 then
        return
    end

    local bin = findDeliveryBin()

```

```

    if not bin then
        return
    end

    bin.reserved = true
    bin.state = 'being_unloaded'
    setStr('cobot_bin_id', tostring(bin.id))
    setInt('req_cobot', 1)

    print(string.format('Coordinator: requested Cobot for bin %d', bin.id))
end

function handleCobotDone()
    if getInt('cobot_done') == 0 then
        return
    end

    local binIdStr = getStr('cobot_bin_id')
    local binId = tonumber(binIdStr)
    local bin = getBinById(binId)

    if bin then
        local color = bin.color
        local remain = getInt(remainSignal(color))
        local delivered = math.min(remain, bin.qty)

        setInt(remainSignal(color), remain - delivered)

        print(string.format(
            'Coordinator: Cobot delivered %d %s part(s) from bin %d',
            delivered, color, bin.id
        ))

        bin.qty = 0
        bin.location = 'producer'
        bin.state = 'empty'
        bin.reserved = false
    else
        print('Coordinator: Cobot done received, but cobot_bin_id was invalid')
    end

    setInt('req_cobot', 0)
    setInt('cobot_done', 0)
    setStr('cobot_bin_id', "")
end

-- =====
-- ORDER COMPLETION
-- =====

function maybeCompleteOrder()
    if not activeOrder then
        return
    end

    if allRemainZero() and not anyRobotBusyOrRequested() then
        setInt('order_complete', 1)
    end
end

```

```

    setInt('order_ready', 0)
    activeOrder = false
    print('Coordinator: order complete')
end
end

-- =====
-- MAIN LOOP
-- =====

function sysCall_actuation()
    local enabled = getInt('system_enabled')

    -- Clean reset when system is turned off
    if enabled == 0 then
        if lastSystemEnabled == 1 then
            print('Coordinator: system disabled, resetting runtime state')
            resetCoordinatorRuntime(false)
        end
        lastSystemEnabled = 0
        return
    end

    lastSystemEnabled = 1

    startOrderIfNeeded()

    if not activeOrder then
        return
    end

    -- Handle completions first
    handlePickPlaceDone()
    handleMobile1Done()
    handleMobile2Done()
    handleCobotDone()

    -- Then issue new work
    maybeRequestPickPlace()
    maybeRequestMobile1()
    maybeRequestMobile2()
    maybeRequestCobot()

    maybeCompleteOrder()
end

```

11. Order Console (for Reference)

```

function sysCall_init()
    sim = require('sim')
    simUI = require('simUI')

    AUTO_START = true
    autoStartPending = false

    orderData = {
        green = 1,
        blue = 1,
        yellow = 1,
    }
end

```

```

        red = 1
    }

    xml = [[
        <ui title="Part Request Console" closeable="false" resizable="false"
activate="false">
            <label text="Enter requested quantities:" />
            <group layout="form" flat="true">
                <label text="Green parts" />
                <edit id="1" value="1" on-editing-finished="greenChanged" />

                <label text="Blue parts" />
                <edit id="2" value="1" on-editing-finished="blueChanged" />

                <label text="Yellow parts" />
                <edit id="3" value="1" on-editing-finished="yellowChanged" />

                <label text="Red parts" />
                <edit id="4" value="1" on-editing-finished="redChanged" />
            </group>

            <group layout="hbox" flat="true">
                <button text="Submit Order" on-click="submitOrder" />
                <button text="Reset" on-click="resetOrder" />
            </group>

            <label text="Status: waiting for input" id="10" />
        </ui>
    ]]

    ui = simUI.create(xml)

    resetAllSignals()
    publishOrder()

    if AUTO_START and totalRequested() > 0 then
        autoStartPending = true
    end
end

function setInt(name, value)
    sim.setInt32Signal(name, value)
end

function setStr(name, value)
    sim.setStringSignal(name, value or '')
end

function getInt(name)
    return sim.getInt32Signal(name) or 0
end

function clear12BinDiagnostics()
    for i = 1, 12 do
        setInt('bin_' .. i .. '_qty', 0)
        setStr('bin_' .. i .. '_color', '')
        setStr('bin_' .. i .. '_state', '')
        setStr('bin_' .. i .. '_location', '')
        setInt('bin_' .. i .. '_reserved', 0)
    end
end

function resetAllSignals()

```

```

    setInt('order_green', 0)
    setInt('order_blue', 0)
    setInt('order_yellow', 0)
    setInt('order_red', 0)

    setInt('remain_green', 0)
    setInt('remain_blue', 0)
    setInt('remain_yellow', 0)
    setInt('remain_red', 0)

    setInt('order_ready', 0)
    setInt('order_complete', 0)
    setInt('system_enabled', 0)

    setInt('req_pick_place', 0)
    setInt('pick_place_done', 0)
    setInt('pick_place_busy', 0)
    setStr('pick_place_color', '')
    setStr('pick_place_bin_id', '')
    setStr('last_picked_color', '')

    setInt('part_ready_at_pick', 0)
    setStr('detected_part_color', '')
    setInt('reject_part', 0)

    setInt('req_mobile1', 0)
    setInt('mobile1_done', 0)
    setInt('mobile1_busy', 0)
    setStr('mobile1_bin_id', '')

    setInt('req_mobile2', 0)
    setInt('mobile2_done', 0)
    setInt('mobile2_busy', 0)
    setStr('mobile2_bin_id', '')

    setInt('req_cobot', 0)
    setInt('cobot_done', 0)
    setInt('cobot_busy', 0)
    setStr('cobot_bin_id', '')

    clear12BinDiagnostics()
end

function publishOrder()
    setInt('order_green', orderData.green)
    setInt('order_blue', orderData.blue)
    setInt('order_yellow', orderData.yellow)
    setInt('order_red', orderData.red)
end

function safeToInt(v)
    local n = tonumber(v)
    if n == nil then
        return 0
    end
    if n < 0 then
        n = 0
    end
    return math.floor(n)
end

function greenChanged(uiHandle, id, newVal)
    orderData.green = safeToInt(newVal)

```



```

end

function blueChanged(uiHandle, id, newVal)
    orderData.blue = safeToInt(newVal)
end

function yellowChanged(uiHandle, id, newVal)
    orderData.yellow = safeToInt(newVal)
end

function redChanged(uiHandle, id, newVal)
    orderData.red = safeToInt(newVal)
end

function totalRequested()
    return orderData.green + orderData.blue + orderData.yellow + orderData.red
end

function submitOrder(uiHandle, id)
    if getInt('order_ready') == 1 and getInt('order_complete') == 0 then
        simUI.setLabelText(ui, 10, 'Status: order already active')
        print('Submit rejected: order already active')
        return
    end

    if totalRequested() == 0 then
        simUI.setLabelText(ui, 10, 'Status: enter at least one quantity')
        print('Submit rejected: no parts requested')
        return
    end

    publishOrder()

    setInt('order_complete', 0)
    setInt('order_ready', 1)
    setInt('system_enabled', 1)

    local msg = string.format(
        'Status: submitted G=%d B=%d Y=%d R=%d',
        orderData.green, orderData.blue, orderData.yellow, orderData.red
    )
    simUI.setLabelText(ui, 10, msg)
    print(msg)
end

function resetOrder(uiHandle, id)
    orderData.green = 0
    orderData.blue = 0
    orderData.yellow = 0
    orderData.red = 0

    resetAllSignals()
    publishOrder()

    simUI.setEditValue(ui, 1, '0', true)
    simUI.setEditValue(ui, 2, '0', true)
    simUI.setEditValue(ui, 3, '0', true)
    simUI.setEditValue(ui, 4, '0', true)
    simUI.setLabelText(ui, 10, 'Status: reset')

    print('Order reset')
end

```

```
function sysCall_sensing()
    if not ui then
        return
    end

    if autoStartPending then
        autoStartPending = false
        submitOrder(nil, nil)
    end

    if getInt('order_complete') == 1 then
        simUI.setLabelText(ui, 10, 'Status: order complete')
    end
end

function sysCall_cleanup()
    if ui then
        simUI.destroy(ui)
    end
end
```